EE2211 Introduction to Machine Learning

Lecture 5

Shaojing Fan
fanshaojing@nus.edu.sg

Office hour (hybrid): Friday 10.30-11.30am, E4-05-26 (Weeks 5-10)

© Copyright EE, NUS. All Rights Reserved.

Course Contents

- Introduction and Preliminaries (Yueming)
  - Introduction
  - Data Engineering
  - Introduction to Probability, Statistics, and Matrix

Module I

- Fundamental Machine Learning Algorithms I (Shaojing)
  - Systems of linear equations
  - Linear regression (least squares regression method)
  - Ridge regression, Polynomial regression

Module II

- Fundamental Machine Learning Algorithms II (Shaojing)
  - Over-fitting, bias/variance trade-off
  - Optimization, Gradient descent
  - Decision Trees, Random Forest
- Performance and More Algorithms (Yueming)
  - Performance Issues
  - K-means Clustering
  - Neural Networks

© Copyright EE, NUS. All Rights Reserved.

Overview of Modules II & III

Lecs 4 & 5: Linear regression 📈 – our very first machine learning model!

Lec 6: Polynomial regression 🌈 – our second model, a more powerful extension of linear regression

Lecs 6-7: Key challenges in model training ⚠️, and strategies to build better models

Lec 8: Gradient descent 🚀 – the engine that powers model training and modern AI!

Lec 9: Decision Tree 🌳 – an intuitive and more explainable model

Neural networks and polynomial regression are powerful—but why does linear regression still matter?

A key principle in ML: the *simplest* model that works is often the best.

© Copyright EE, NUS. All Rights Reserved.

Recap: Solutions of L.E.S (Based on RC Theorem)

For $\mathbf{X} \in \mathcal{R}^{m\times d}$ and $\widetilde{\mathbf{X}} = [\mathbf{X}\ \mathbf{y}] \in \mathcal{R}^{m\times(d+1)}$, the system $\mathbf{X}\mathbf{w} = \mathbf{y}$

	Even (m = d)	Over (m > d)	Under (m < d)
i) $\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) = d$	Unique sol: $\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$	Unique sol: $\hat{\mathbf{w}} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}$	--
ii) $\text{rank}(\mathbf{X}) < \text{rank}(\widetilde{\mathbf{X}})$	No sol	No sol in general, but when left inv $\mathbf{X}$ exists, least squares sol (approx.): $\hat{\mathbf{w}} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}$	No sol
iii) $\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) < d$	Infinitely many sol*	Infinitely many sol*	Infinitely many sol in general, but when right inv $\mathbf{X}$ exists, least norm sol: $\hat{\mathbf{w}} = \mathbf{X}^{\mathrm{T}}(\mathbf{X}\mathbf{X}^{\mathrm{T}})^{-1}\mathbf{y}$

*These particular cases are not be covered in this course.

© Copyright EE, NUS. All Rights Reserved.

Questions from Lecture 4

Qn 1: For an over-determined system, which generally has no exact solution. On the exam, when asked to solve for $\mathbf{w}$, should we just write "no solution" or provide the least-squares solution?

An: We should provide a least-squares solution.

What Makes Machine Learning Different?

- **Error Tolerant:** Some error is inevitable in ML. ML engineers usually look for an **approximate solution** that allows some error.
- **Probabilistic Thinking:** What ML does is to provide the best estimate and the most likely outcome.

In ML, we speak in terms of *probabilities* rather than *deterministic answers.*

© Copyright EE, NUS. All Rights Reserved.

Questions from Lecture 4

Qn 2: For an under-determined system, which generally has infinitely many solutions, can we just pick one at random instead of computing the least-norm solution?

An: Mathematically, we *can* pick any solution, but in the **ML context** it is **not recommended.**

In ML, picking a random solution is dangerous: it may have huge weights that cause extreme sensitivity to noise in new data, leading to poor generalization. The least-norm solution is preferred because it's typically more stable and generalizes better.

© Copyright EE, NUS. All Rights Reserved.

Questions from Lecture 4

Example: House price prediction, 2 features, only one sample

$$Price = w_1 \cdot (house\ size) + w_2 \cdot (\#\ of\ bedrooms\ )$$

Data for training:

House	House size in tsf (x_1)	# of bedrooms (x_2)	Actual Sold Price (y)
House A	3	4	$1,200,000

So our single equation is: $3 \cdot w_1 + 4 \cdot w_2 = 1{,}200{,}000$

- **Solution 1** (only house size matters): $w_1 = 400{,}000,\ \ w_2 = 0$
- **Solution 2** (only bedrooms matters): $w_1 = 0,\ w_2 = 300{,}000$

Data for prediction:

House B: Size = 2.5 (tsf), # of bedrooms: 5

If we use solution 2: predicted sold price $= 2.5 \cdot 0 + 5 \cdot 300{,}000 = 1{,}500{,}000$

One extra bedroom, smaller house size, → **$300,000**↑ (unrealistic)

- **Solution 3** (minimize $||w||^2 = {w_1}^2 + {w_2}^2$): w_1 = 144,000, w_2 = 192,000

The least-norm solution is preferred because it inherently avoids extreme weights, resulting in a more *stable model* that *generalizes better to unseen data*.

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Set and functions
- Derivative and gradient
- Linear regression
- Linear regression with multiple outputs

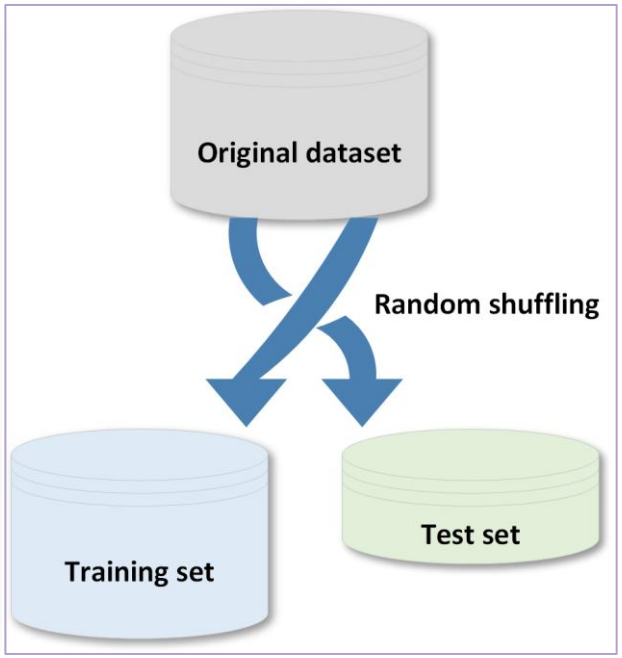
© Copyright EE, NUS. All Rights Reserved.

Set: Motivation

Before we talk about machine learning, we need a language to talk about “collections of things”. That language is **set**.

In machine learning, a data set* refers to a collection of samples, which may be used as training data, test data, or both.

17 Nov 2025 Blk 37 #09-XX	$1,188 psf 1,389 sqft	$1.650 M
14 Nov 2025 Blk 43 #04-XX	$1,214 psf 1,227 sqft	$1.490 M
04 Nov 2025 Blk 41 #02-XX	$1,236 psf 1,076 sqft	$1.330 M
27 Oct 2025 Blk 35 #16-XX	$1,332 psf 1,389 sqft	$1.850 M
29 Sep 2025 Blk 35 #03-XX	$1,180 psf 1,227 sqft	$1.448 M



* In machine learning, “dataset” and “data set” are often used interchangeably

© Copyright EE, NUS. All Rights Reserved.

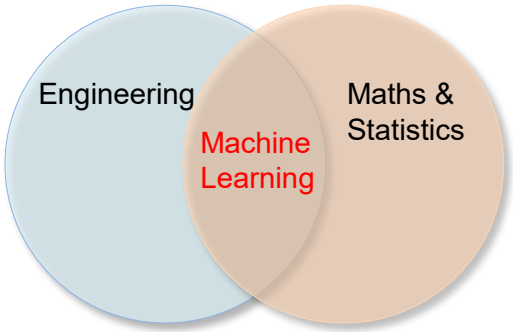
Notations: Set

- Mathematically, a **set** $\mathcal{S}$ is an **unordered** collection of **unique** elements

➢ Example:

- $\mathcal{S} = \{1, 2, 3, 4, 5, 6\}$ is the possible outcomes of the toss of a die.
- $\mathbb{R}$ is the set of all real numbers.
- $\mathbb{R}^d$ is the set of all real vectors of length d

In a ML context, a data set can often contain **duplicate** samples, so strictly speaking, it's not a mathematical set. In practice, however, the term "set" is often used loosely in machine learning to mean a collection of data points.



Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (p4 of chp2).

© Copyright EE, NUS. All Rights Reserved.

Functions: Definition

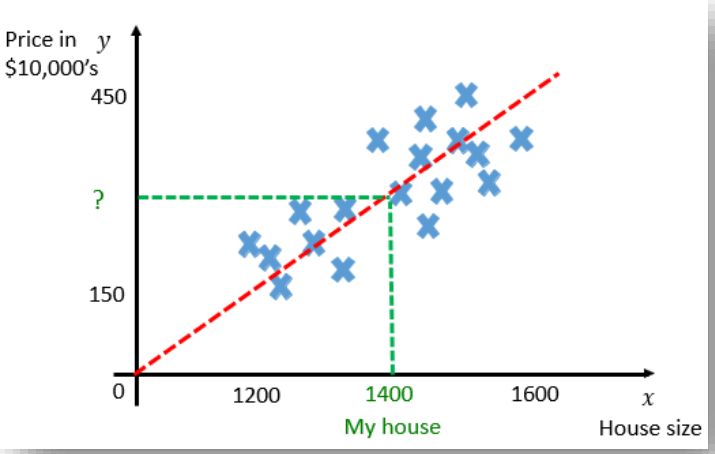
- A function f is a map from a set X to another set Y. We write this as:

$$f: X \to Y$$

- For example, the function $f: \mathbb{R} \to [0, \infty]$ could be given by the recipe $f(x) = x^2$.

- The set of inputs is called the domain; the set of possible outputs is called the codomain; the set $\{f(x) : x \in X\}$ is called the range.

A machine learning model is just a special kind of function that learns to map inputs to outputs, based on the data we have.



Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (p6 of chp2).

© Copyright EE, NUS. All Rights Reserved.

Linear Functions

A function $f: \mathcal{R}^d \to \mathcal{R}$ is **linear** if it satisfies the following two properties:

- **Homogeneity**
  - For any *d*-vector $\mathbf{x}$ and any scalar α, $f(\alpha\mathbf{x}) = \alpha f(\mathbf{x})$
  - *Scaling the (vector) argument is the same as scaling the function value*

- **Additivity**
  - For any *d*-vectors $\mathbf{x}$ and $\mathbf{y}$, $f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$
  - *Adding (vector) arguments is the same as adding the function values*

For linear functions, scaling and addition can be moved from the input to the output without changing the result.

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, "Introduction to Applied Linear Algebra", Cambridge University Press, 2018 (p31)

© Copyright EE, NUS. All Rights Reserved.

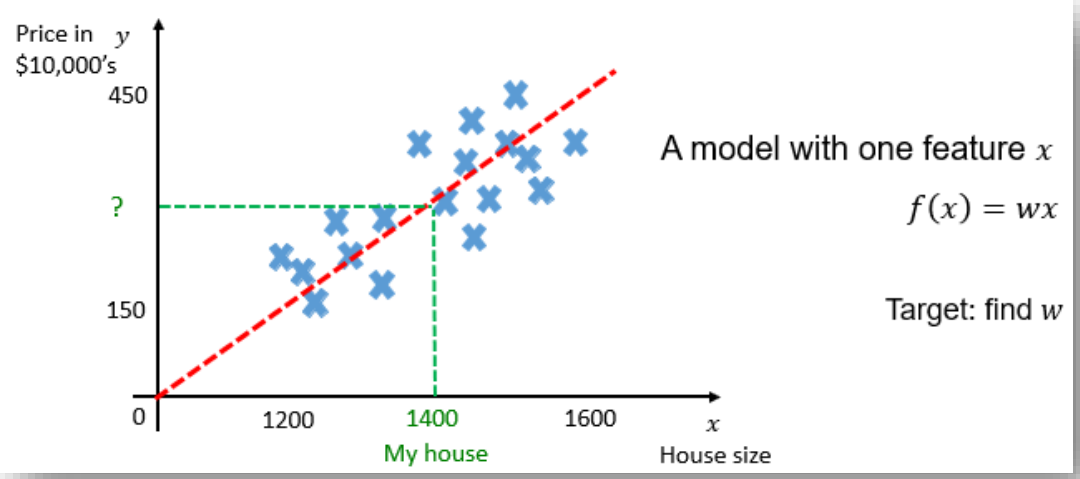
Linear Functions: Inner Product Function

Fix a vector $\mathbf{w} \in \mathcal{R}^d$ and define the function $f: \mathcal{R}^d \to \mathcal{R}$ as

$$f(\mathbf{x}) = \mathbf{w}^{\mathbf{T}}\mathbf{x} = \sum_{i=1}^{d} w_i x_i = w_1 x_1 + w_2 x_2 + \cdots w_d x_d$$

This is called the inner product function.

E.g., simple machine learning models, such as linear regression, are built on the inner product between a weight vector and an input vector ($f(\mathbf{x}) = \mathbf{w}^{\mathbf{T}}\mathbf{x}$).



The inner product function is **a linear function** because it is a **weighted sum** of the input components, which satisfies *additivity* and *homogeneity*.

© Copyright EE, NUS. All Rights Reserved.

Quick Question 1

True or False: A linear function $f: \mathcal{R}^d \to \mathcal{R}$ must pass through the origin. That is, $f(\mathbf{0}) = 0$ where $\mathbf{0} \in \mathcal{R}^d$ is the zero vector in d dimensions.

PollEv.com/shaojing

A function $f: \mathcal{R}^d \to \mathcal{R}$ is **linear** if it satisfies the following two properties:

- **Homogeneity**
  - For any d-vector $\mathbf{x}$ and any scalar α, $f(\alpha\mathbf{x}) = \alpha f(\mathbf{x})$
- **Additivity**
  - For any d-vectors $\mathbf{x}$ and $\mathbf{y}$, $f(\mathbf{x}+\mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$

© Copyright EE, NUS. All Rights Reserved.

Quick Question 1 Takeaway

True or False: A linear function $f: \mathcal{R}^d \to \mathcal{R}$ must pass through the origin. That is, $f(\mathbf{0}) = 0$ where $\mathbf{0} \in \mathcal{R}^d$ is the zero vector in d dimensions. ✓

Suppose we have a function with an offset b:

$$f(x) = wx + b$$

Any offset (constant term) breaks (strict) linearity

Let's test **homogeneity** with $c = 2$:

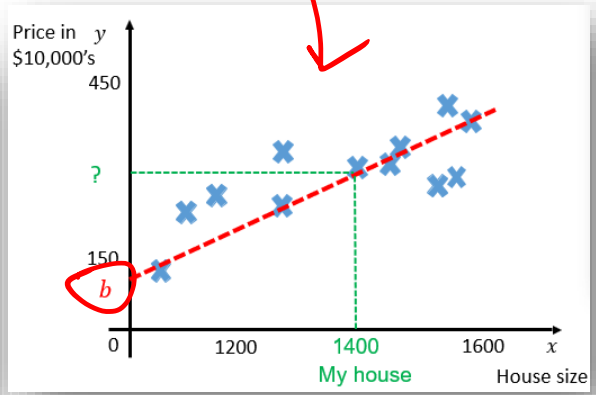
$f(2 \cdot x) = 2wx + b$

$2f(x) = 2wx + 2b$,

Since $b \neq 2b$ when $b \neq 0$, we have $f(2x) \neq 2f(x)$.

A function $f: \mathcal{R}^d \to \mathcal{R}$ is **linear** if it satisfies the following two properties:

- **Homogeneity**
  - For any d-vector $\mathbf{x}$ and any scalar α, $f(\alpha\mathbf{x}) = \alpha f(\mathbf{x})$
- **Additivity**
  - For any d-vectors $\mathbf{x}$ and $\mathbf{y}$, $f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y})$



© Copyright EE, NUS. All Rights Reserved.

Affine Functions

A linear function plus a constant is called an affine function

It can be written as

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$$

where $\mathbf{w}$ is a d-vector, and b is a scalar, called the offset (or bias)

Example:

$$f(\mathbf{x}) = \overbrace{2.3}^{b} - 2x_1 + 1.3x_2 - x_3$$

This function is affine, with $b = 2.3$, $\mathbf{w}^T = [-2,\ 1.3,\ -1]$.

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, "Introduction to Applied Linear Algebra", Cambridge University Press, 2018 (p32)

© Copyright EE, NUS. All Rights Reserved.

Linear vs Affine Functions

- An **affine function** is **not necessarily a linear function**, but every linear function is a special case of an affine function.

- An **affine function** $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ is a **linear function** if and only if the constant term $b = 0$.

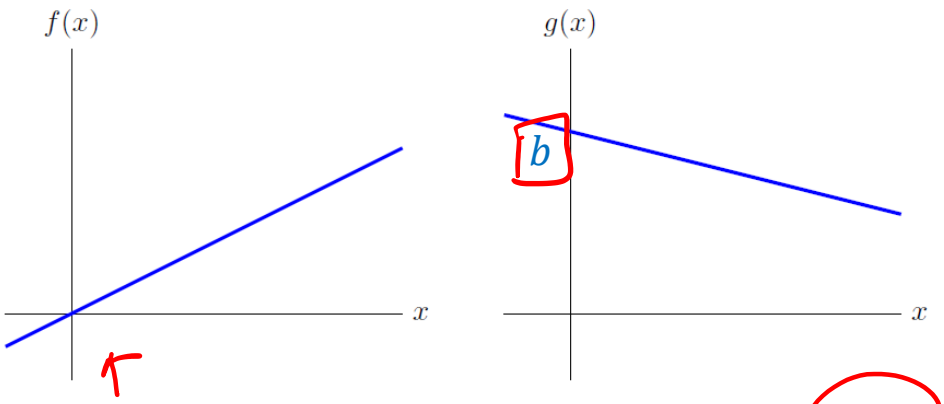
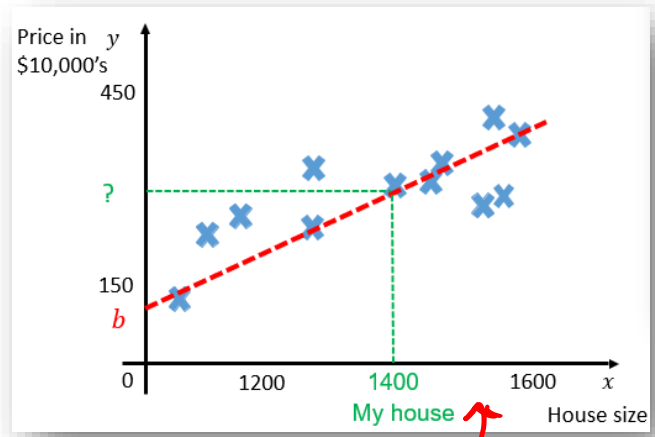


Figure 2.1 *Left.* The function f is linear. *Right.* The function g is affine, but not linear.



In ML and AI, we often use the term "linear" loosely to include both. E.g., the model on the right is commonly referred to as a linear regression model, even though it is mathematically an affine function.

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, "Introduction to Applied Linear Algebra", Cambridge University Press, 2018 (p32)

© Copyright EE, NUS. All Rights Reserved.

Scalar vs Vector Functions

- A **scalar function** is a function whose output is a single number (scalar)

  Formally: $f: \mathcal{R}^d \rightarrow \mathcal{R}$

  - Input: d-dimensional vector (if $d = 1$, equivalent to a real number)
  - Output: a single real value (scalar)

  – E.g. $f(\mathbf{x}) = x_1 + x_2 + 2x_3$; $f(\mathbf{x}) = (x_1 - 1)^2 + (x_2 - 2)^2$

  scalar

  $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$ 3d vector

  $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$

- A **vector function** is a function whose output is a vector

  Formally: $\mathbf{f}: \mathcal{R}^d \rightarrow \mathcal{R}^k$

  - Input: d-dimensional vector (if $d = 1$, equivalent to a real number)
  - Output: a k-dimensional vector (if $k = 1$, it reduces to a scalar function)

  – E.g. $\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -x_1 \\ x_2 \end{bmatrix}$

  – E.g., $f(x_1, x_2) = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 2x_1 + x_2 \\ x_1 - 5x_2 \end{bmatrix}$ $y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (p7 of chp2).

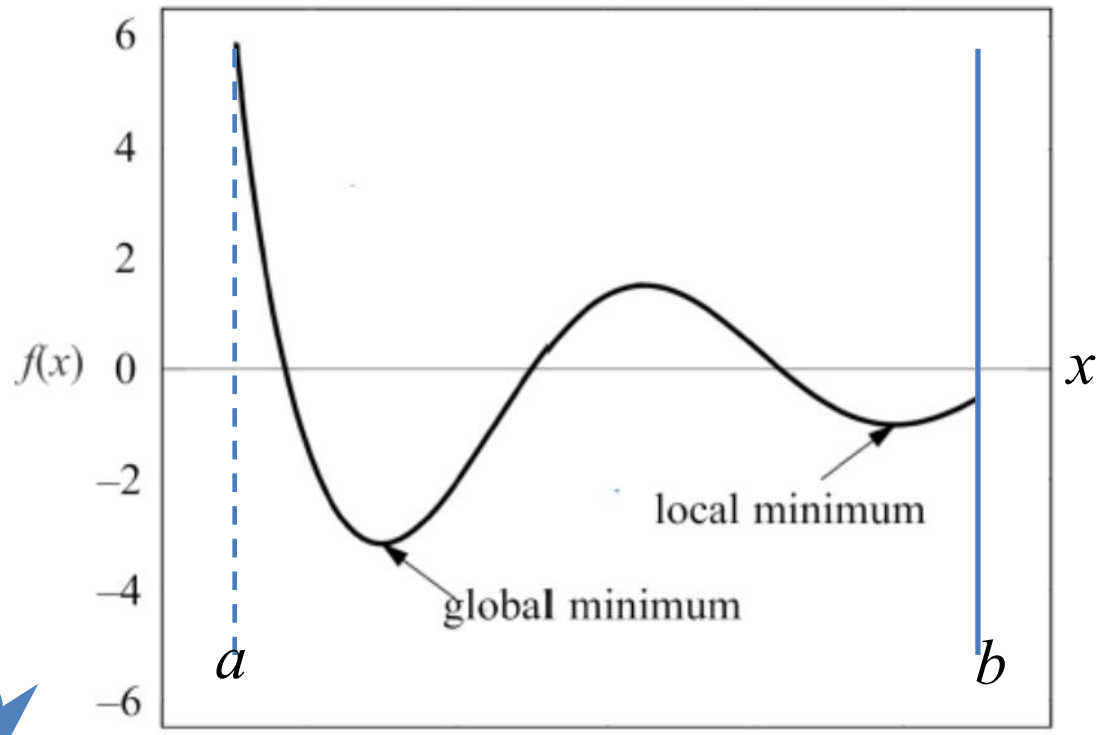
© Copyright EE, NUS. All Rights Reserved.

Functions: Maximum and Minimum

- $f(x)$ has a **local minimum** at $x = c$ if $f(x) \geq f(c)$ for every x in some open interval around $x = c$

- $f(x)$ has a **global minimum** at $x = c$ if $f(x) \geq f(c)$ for all x in the domain of f

Local: "smallest in a *neighborhood*,"
Global: "smallest *everywhere*."



$a < x \leq b$

Note: An interval is a set of real numbers with the property that any number that lies between two numbers in the set is also included in the set.
An open interval does not include its endpoints and is denoted using parentheses. E.g. (0, 1) means "all numbers greater than 0 and less than 1".

Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (p6-7 of chp2).

© Copyright EE, NUS. All Rights Reserved.

Functions: Maximum and Minimum

Max and Arg Max

Given a set of values $\mathcal{A} = \{a_1,\ a_2, \dots,\ a_m\}$,

- To find the maximum of the function $f(a)$ for all a in $\mathcal{A}$

$$\max_{a \in \mathcal{A}} f(a)$$

- To find a in $\mathcal{A}$ that maximizes function $f(a)$

$$\operatorname*{argmax}_{a \in \mathcal{A}} f(a)$$

Max/min return the **function** value; **argmax/argmin** return the **input** value that achieves that maximum or minimum.

E.g. $f(a) = 3a,\ a \in [0,1] \rightarrow \max_{a} f(a) = 3$ and $\operatorname*{argmax}_{a} f(a) = 1$

Quick question: what is $\min_{a \in \mathcal{A}} f(a)$ and $\operatorname*{argmin}_{a \in \mathcal{A}} f(a)$?

Min and **Arg Min** operate in a similar manner:

$$\min_{a \in \mathcal{A}} f(a) \qquad \operatorname*{argmin}_{a \in \mathcal{A}} f(a)$$

Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (p6-7 of chp2).

© Copyright EE, NUS. All Rights Reserved.

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Set and functions
- Derivative and gradient
- Linear regression
- Linear regression with multiple outputs

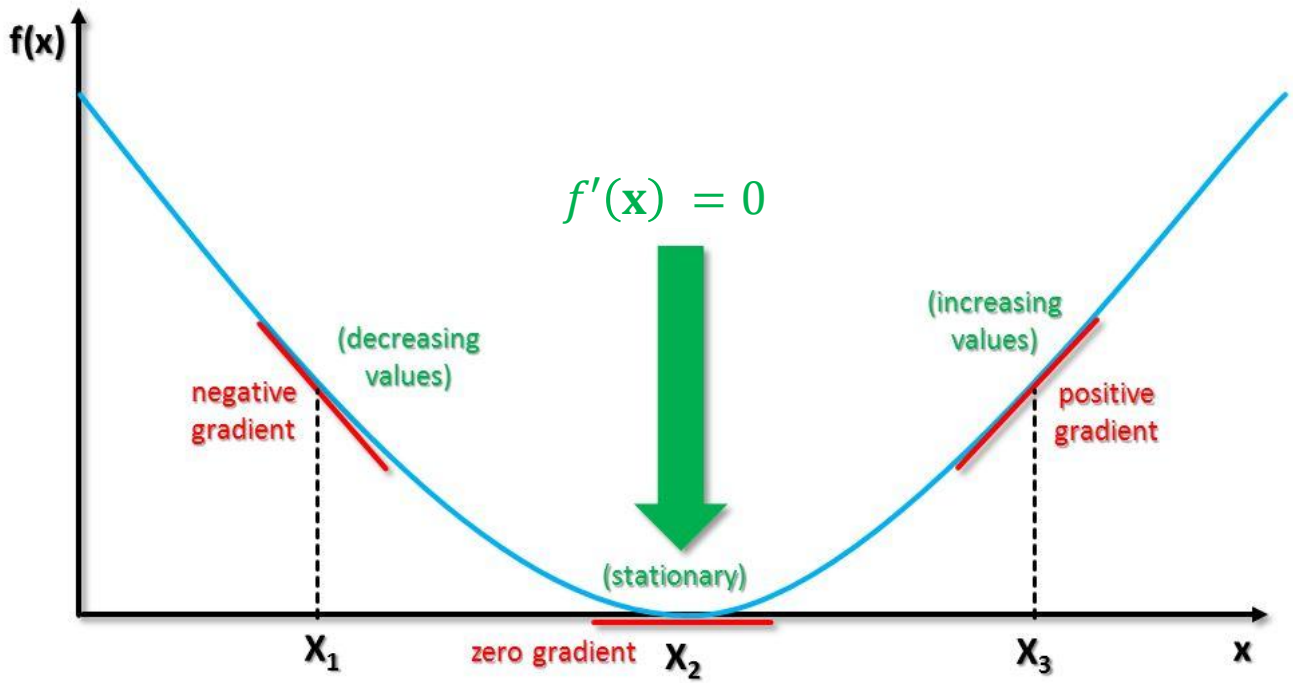
Fundamental tools for learning machine learning models

© Copyright EE, NUS. All Rights Reserved.

Derivative and Gradient

The **derivative** $f'(\mathbf{x})$ of a function $f(\mathbf{x})$ tells how fast the function increases or decreases.

- If $f'(\mathbf{x})$ is constant, the function grows at a *steady* rate.
- If $f'(\mathbf{x}) > 0$, the function is *increasing* at $\mathbf{x}$.
- If $f'(\mathbf{x}) < 0$, the function is *decreasing* at $\mathbf{x}$.
- If $f'(\mathbf{x}) = 0$, the slope is *flat* — possibly a maximum or minimum.

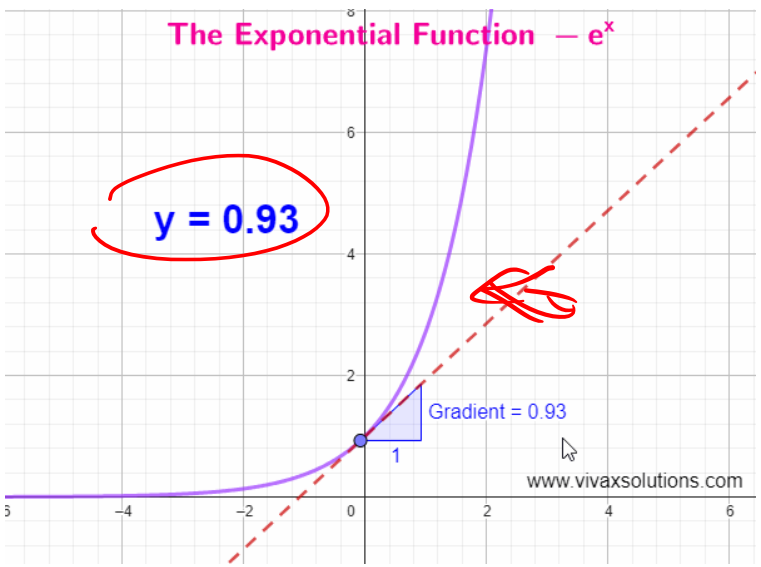


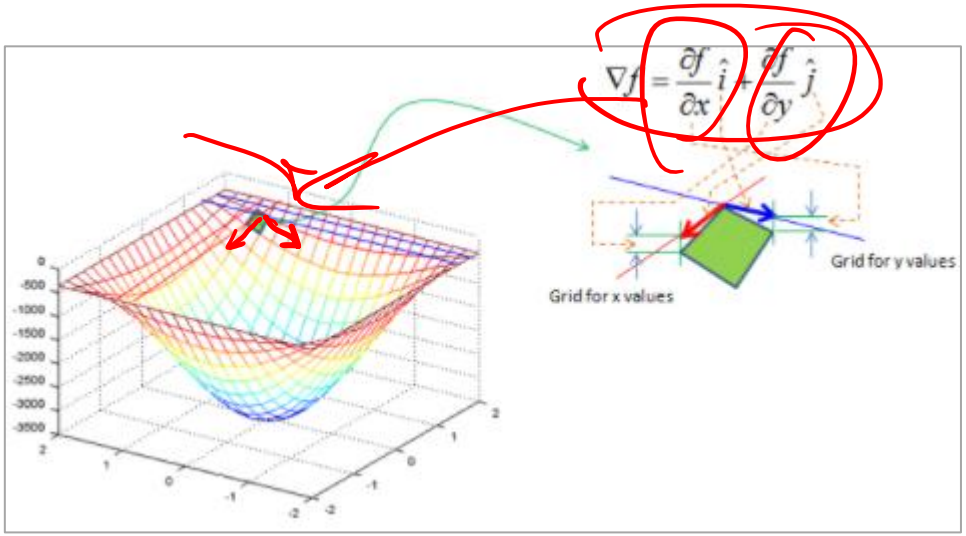
Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (p8 of chp2).

© Copyright EE, NUS. All Rights Reserved.

Derivative and Gradient: Visualisation

- The **derivative** $f'(\mathbf{x})$ of a function $f(\mathbf{x})$ tells how fast the function increases or decreases.
- The process of finding a derivative is called **differentiation.**
- The **gradient** is a *generalization* of the derivative for functions with **multiple inputs** (*i.e.*, vectors).





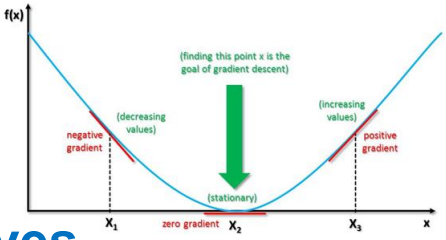
Source: https://blog.vivaxsolutions.com/2024/04/the-exponential-graph.html
https://mathematica.stackexchange.com/questions/299799/creating-a-3d-gradient-vector-field-plot-in-mathematica

Python visualization

All code have been uploaded to Canvas

© Copyright EE, NUS. All Rights Reserved.

Computing Gradient: Case 1



The **gradient** of a function is a vector of **partial derivatives**

Differentiating a scalar function w.r.t. a vector

If $f(\mathbf{x})$ is a scalar function of d variables, $\mathbf{x}$ is a $d \times 1$ vector.
Then differentiation of $f(\mathbf{x})$ w.r.t. $\mathbf{x}$ results in a $d \times 1$ vector

$$\frac{df(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix}$$

Partial derivatives: Focus on one input variable at a time, while treating all other input variables as constants.

A scalar function maps one or more inputs (scalars or vectors) to a *single scalar output*. E.g.

$$f(\mathbf{x}) = 2x_1 - x_2 + 3x_3 + 5$$

This is referred to as the **gradient** of $f(\mathbf{x})$ and often written as $\nabla_{\mathbf{x}} f$.

E.g. $f(\mathbf{x}) = ax_1 + bx_2 \quad \nabla_{\mathbf{x}} f = \begin{bmatrix} a \\ b \end{bmatrix}$

Ref: Duda, Hart, and Stork, "Pattern Classification", 2001 (Appendix)

© Copyright EE, NUS. All Rights Reserved.

Computing Gradient: Case 2

Differentiating of a vector function w.r.t. a vector

If $\mathbf{f}(\mathbf{x})$ is a vector function of *size* $h \times 1$ and $\mathbf{x}$ is a $d \times 1$ vector. Then differentiation of $\mathbf{f}(\mathbf{x})$ results in a $h \times d$ matrix

$$\frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_h}{\partial x_1} & \cdots & \frac{\partial f_h}{\partial x_d} \end{bmatrix}$$

An $h \times 1$ **vector function** maps one or more inputs (scalars or vectors) to an $h \times 1$ vector output. E.g.,

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} \Leftarrow \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

The matrix is referred to as the **Jacobian** of $\mathbf{f}(\mathbf{x})$

Ref: Duda, Hart, and Stork, "Pattern Classification", 2001 (Appendix)

© Copyright EE, NUS. All Rights Reserved.

Computing Gradient: Example

Example:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \frac{d\mathbf{Ax}}{d\mathbf{x}} = ?$$

$$\boxed{\frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_h}{\partial x_1} & \cdots & \frac{\partial f_h}{\partial x_d} \end{bmatrix}}$$

$$\begin{cases} f_1(x_1, x_2, x_3) = a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \\ f_2(x_1, x_2, x_3) = a_{21}\,x_1 + a_{22}x_2 + a_{23}x \end{cases}$$

2 x 1 vector function

$$\mathbf{f}(\mathbf{x}) = \mathbf{Ax} = \begin{bmatrix} f_1(x_1, x_2, x_3) \\ f_2(x_1, x_2, x_3) \end{bmatrix} = \begin{bmatrix} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 \end{bmatrix}$$

$2 \times 1 \Rightarrow \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$

$$\frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \frac{d\mathbf{Ax}}{d\mathbf{x}} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix} = \mathbf{A}$$

$\mathbf{f}(\mathbf{x})$ is a vector function of size $h \times 1$, $\mathbf{x}$ is a $d \times 1$ vector, then differentiation of $\mathbf{f}(\mathbf{x})$ results in a $h \times d$ matrix.

© Copyright EE, NUS. All Rights Reserved.

Computing Gradient: Common Formulae

Some Vector-Matrix Differentiation Formulae:

$$\frac{d\mathbf{A}\mathbf{x}}{d\mathbf{x}} = \mathbf{A}$$

E.g., $f(\mathbf{x}) = \mathbf{a}^T\mathbf{x} = a_1x_1 + a_2x_2 + \cdots a_dx_d$

$$\frac{d(\mathbf{a}^T\mathbf{x})}{d\mathbf{x}} = \mathbf{a}$$

$$\frac{df(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_d \end{bmatrix} = \mathbf{a}$$

$$\frac{d(\mathbf{x}^T\mathbf{A}\mathbf{x})}{d\mathbf{x}} = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$$

Derivations: https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

Ref: Duda, Hart, and Stork, “Pattern Classification”, 2001 (Appendix)

© Copyright EE, NUS. All Rights Reserved.

Computing Gradient: Common Formulae

$$\frac{d(\mathbf{x}^T\mathbf{A}\mathbf{x})}{d\mathbf{x}} = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$$

Product rule: $d(uv) = (du)v + u(dv)$

$$\frac{d(\mathbf{x}^T\mathbf{A}\mathbf{x})}{d\mathbf{x}} = \frac{d(\mathbf{x}^T\mathbf{A})}{d\mathbf{x}}\mathbf{x} + \mathbf{x}^T\mathbf{A}\frac{d(\mathbf{x})}{d\mathbf{x}} = \frac{d(\mathbf{x}^T\mathbf{A})}{d\mathbf{x}}\mathbf{x} + \mathbf{x}^T\mathbf{A}\mathbf{I} = \mathbf{A}^T\mathbf{x} + \mathbf{A}\mathbf{x} = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$$

Note: The above derivation is provided for reference only and is not required for this class.

Derivations: https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf

Ref: Duda, Hart, and Stork, “Pattern Classification”, 2001 (Appendix)

© Copyright EE, NUS. All Rights Reserved.

Quick Question 2

Let $\mathbf{f}(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2 \\ 3x_1 - x_2^2 \end{bmatrix}$, $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. What is the differentiation of $\mathbf{f}(\mathbf{x})$ w.r.t. $\mathbf{x}$?

$\rightarrow 2 \times 1$

$\nabla_x f(x) \rightarrow 2 \times 2$

$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$ 2×1

A. $\begin{bmatrix} 2x_1 + 1 \\ 3 - 2x_2 \end{bmatrix}$ $\times \leftarrow \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \rightarrow 2 \times 1$

B. $\begin{bmatrix} 2x_1 & 1 \\ 3 & -2x_2 \end{bmatrix}$

C. $\begin{bmatrix} x_1^2 & x_2 \\ 3x_1 & -x_2^2 \end{bmatrix}$

D. $\begin{bmatrix} 2 & 1 \\ 3 & -2 \end{bmatrix}$

PollEv.com/shaojing

© Copyright EE, NUS. All Rights Reserved.

Computing gradient

0

Let $\mathbf{f}(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2 \\ 3x_1 - x_2^2 \end{bmatrix}$, $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. What is the differentiation of $\mathbf{f}(\mathbf{x})$ w.r.t. $\mathbf{x}$?

A. $\begin{bmatrix} 2x_1 + 1 \\ 3 - 2x_2 \end{bmatrix}$

B. $\begin{bmatrix} 2x_1 & 1 \\ 3 & -2x_2 \end{bmatrix}$

C. $\begin{bmatrix} x_1^2 & x_2 \\ 3x_1 & -x_2^2 \end{bmatrix}$

D. $\begin{bmatrix} 2 & 1 \\ 3 & -2 \end{bmatrix}$

A

0%

B

0%

C

0%

D

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

© Copyright EE, NUS. All Rights Reserved.

Quick Question 2

Let $\mathbf{f}(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2 \\ 3x_1 - x_2^2 \end{bmatrix}$, $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. What is the differentiation of $\mathbf{f}(\mathbf{x})$ w.r.t. $\mathbf{x}$?

A. $\begin{bmatrix} 2x_1 + 1 \\ 3 - 2x_2 \end{bmatrix}$

B. $\begin{bmatrix} 2x_1 & 1 \\ 3 & -2x_2 \end{bmatrix}$

$\frac{\partial f_1}{\partial x_1}$ $\frac{\partial f_1}{\partial x_2}$ $\frac{\partial f_2}{\partial x_2}$

C. $\begin{bmatrix} x_1^2 & x_2 \\ 3x_1 & -x_2^2 \end{bmatrix}$

D. $\begin{bmatrix} 2 & 1 \\ 3 & -2 \end{bmatrix}$

If $\mathbf{f}(\mathbf{x})$ is a vector function of *size* $h \times 1$ and $\mathbf{x}$ is a $d \times 1$ vector. Then differentiation of $\mathbf{f}(\mathbf{x})$ results in a $h \times d$ matrix

$$\frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_h}{\partial x_1} & \cdots & \frac{\partial f_h}{\partial x_d} \end{bmatrix}$$

- $\mathbf{f}(\mathbf{x})$ is a 2×1 vector function
- $\mathbf{x}$ is a 2×1 vector
- Differentiation of $\mathbf{f}(\mathbf{x})$ w.r.t. $\mathbf{x}$ produces a 2×2 matrix
- Each entry is a partial derivative: $\frac{\partial f_i}{\partial x_j}$

© Copyright EE, NUS. All Rights Reserved.

© Copyright EE, NUS. All Rights Reserved.

Agenda

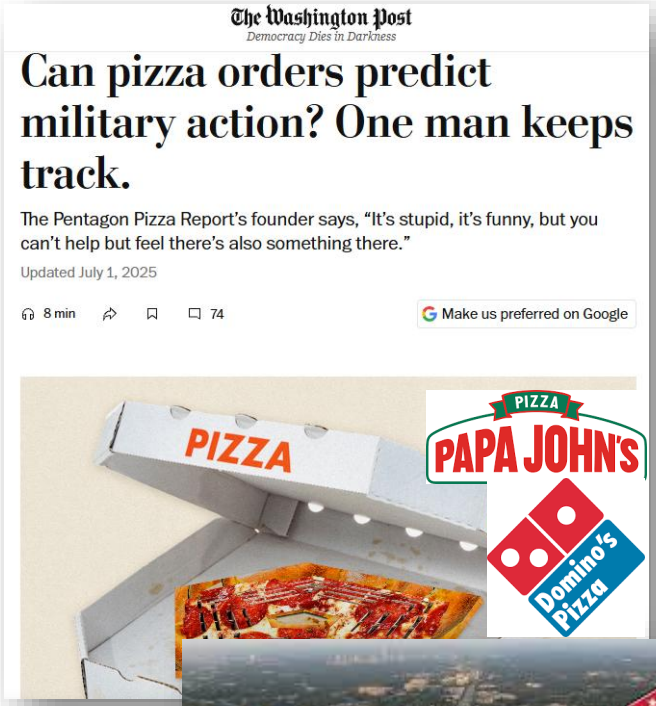
- Set and functions
- Derivative and gradient
- Linear regression
- Linear regression with multiple outputs

A key principle in ML: the simplest model that works is often the best.

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Motivation

The "Pentagon Pizza Index"


The Washington Post
Democracy Dies in Darkness

Can pizza orders predict military action? One man keeps track.

The Pentagon Pizza Report's founder says, "It's stupid, it's funny, but you can't help but feel there's also something there."

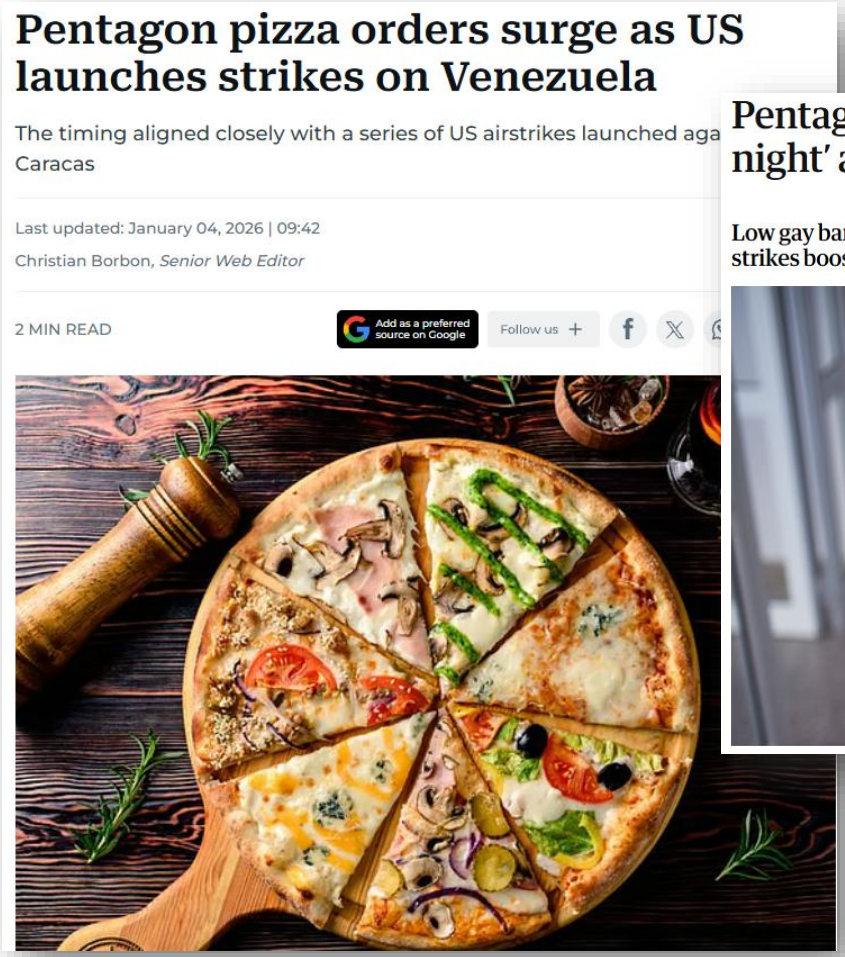
Updated July 1, 2025

8 min 74 Make us preferred on Google

Reference: https://en.wikipedia.org/wiki/Pentagon_pizza_theory
Note: This is a fun pedagogical example used for motivation, but it is not a scientifically validated model.

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Motivation



Pentagon pizza orders surge as US launches strikes on Venezuela

The timing aligned closely with a series of US airstrikes launched aga
Caracas

Last updated: January 04, 2026 | 09:42

Christian Borbon, *Senior Web Editor*

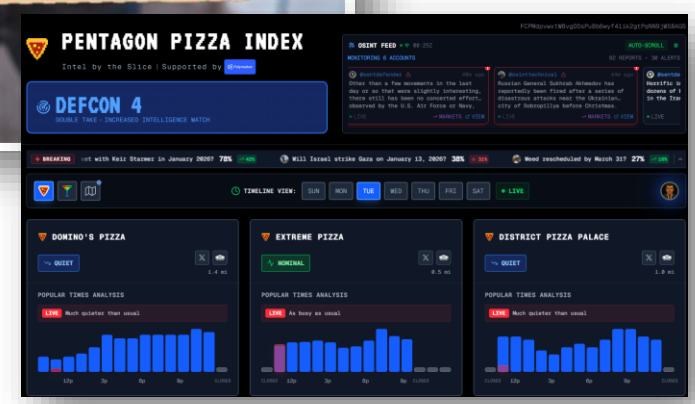
2 MIN READ

Add as a preferred source on Google

Follow us +

Pentagon pizza monitor predicted 'busy night' ahead of Israel's attack on Iran

Low gay bar traffic and spike in pizza deliveries before strikes boost theory on Pentagon activity during invasions



Source: https://gulfnews.com/world/offbeat/pentagon-pizza-orders-surge-as-us-launches-strikes-on-venezuela-1.500397948
https://www.theguardian.com/world/2025/jun/13/pentagon-pizza-delivery-israel-iran-attack

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Concept

- **Linear regression** is a popular regression learning algorithm that learns a model which is a linear combination of features of the input example.

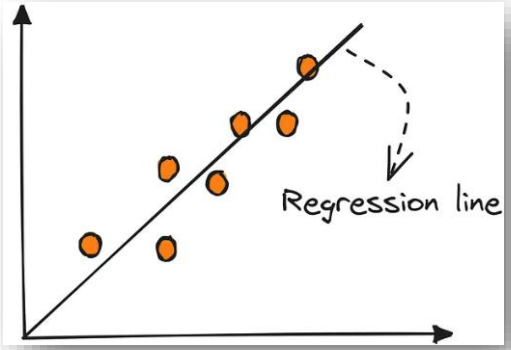
$$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m\times d}, \mathbf{w} \in \mathcal{R}^{d}, \mathbf{y} \in \mathcal{R}^{m}$$

feaer ↓ ↓ ? label

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m,1} & x_{m,2} & \cdots & x_{m,d} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

sample

- The data matrix $\mathbf{X} \in \mathcal{R}^{m\times d}$ and the target vector $\mathbf{y} \in \mathcal{R}^{m}$ are given
- The unknown vector of parameters $\mathbf{w} \in \mathcal{R}^{d}$ is to be learnt.

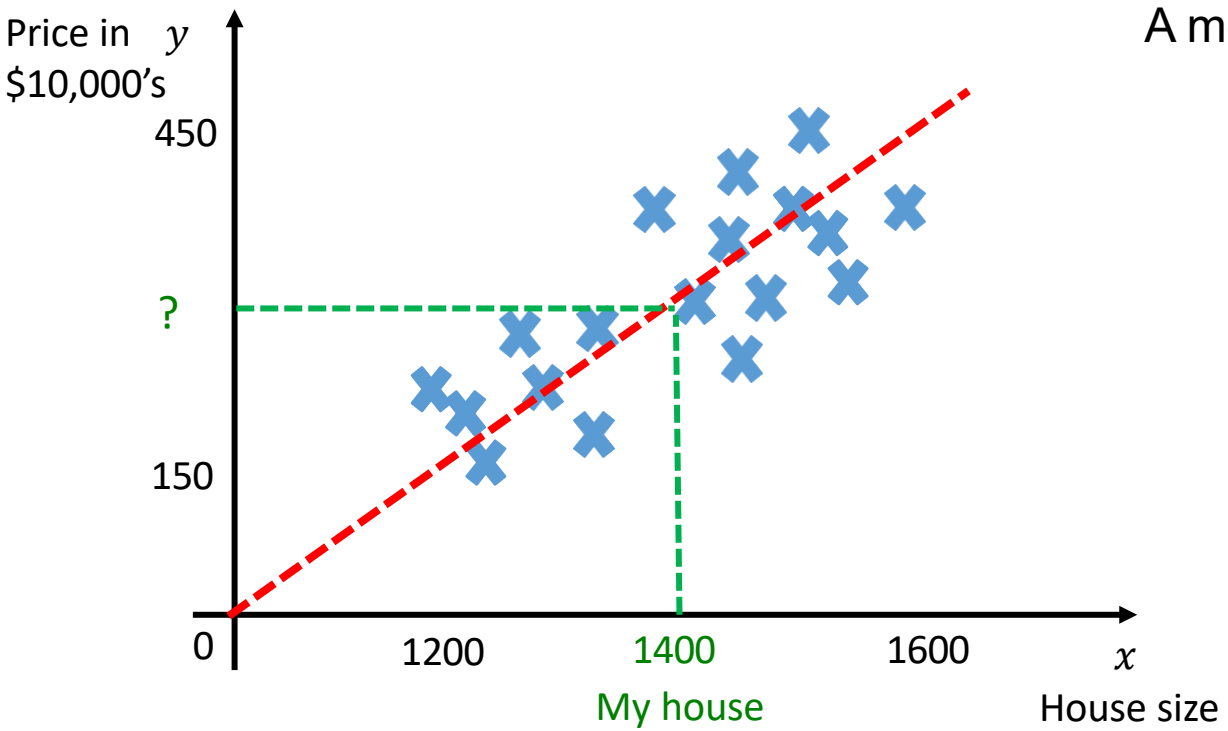


Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (p3 of chp3).

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example

Example: I want to sell my 1,400 ft² house—how much should I ask for?



A model with one feature x

$$f(x) = wx$$

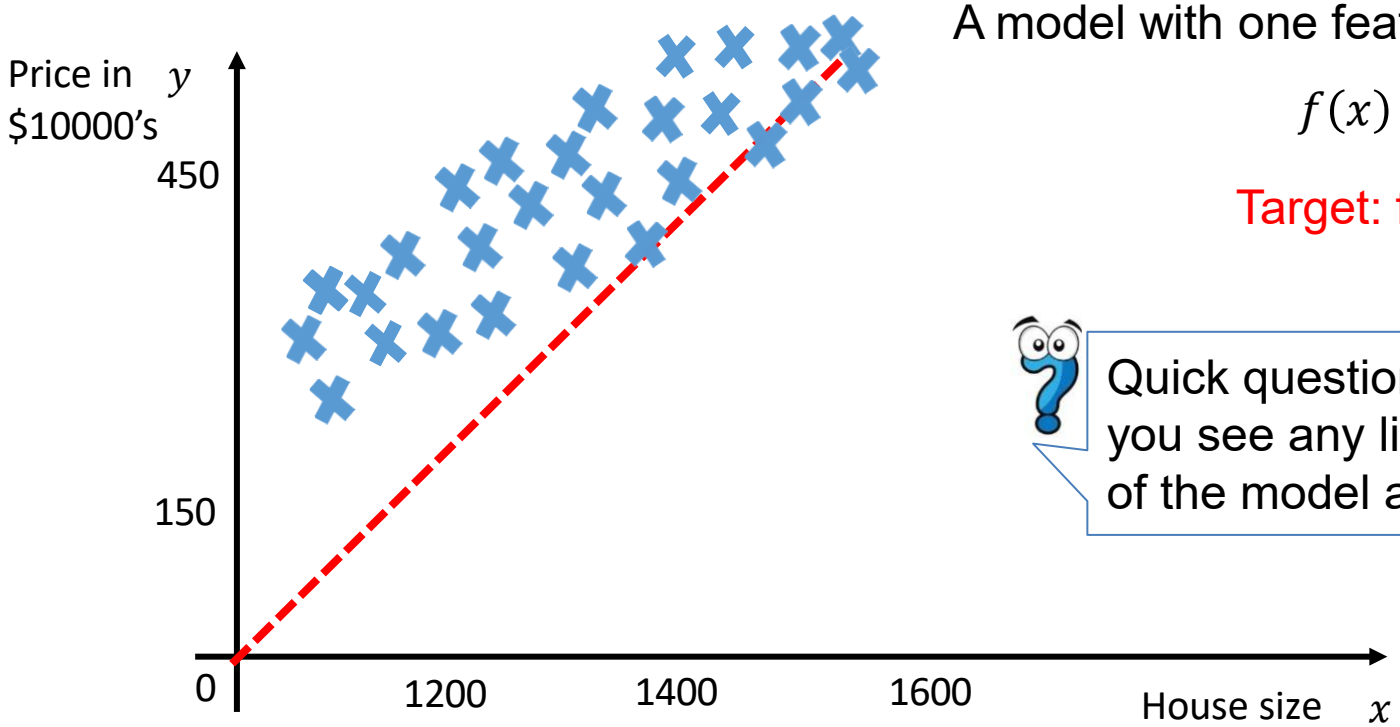
Target: find w

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example

Example: I want to sell my 1,400 ft² house—how much should I ask for?

More practical scenario case:



A model with one feature x

$$f(x) = wx$$

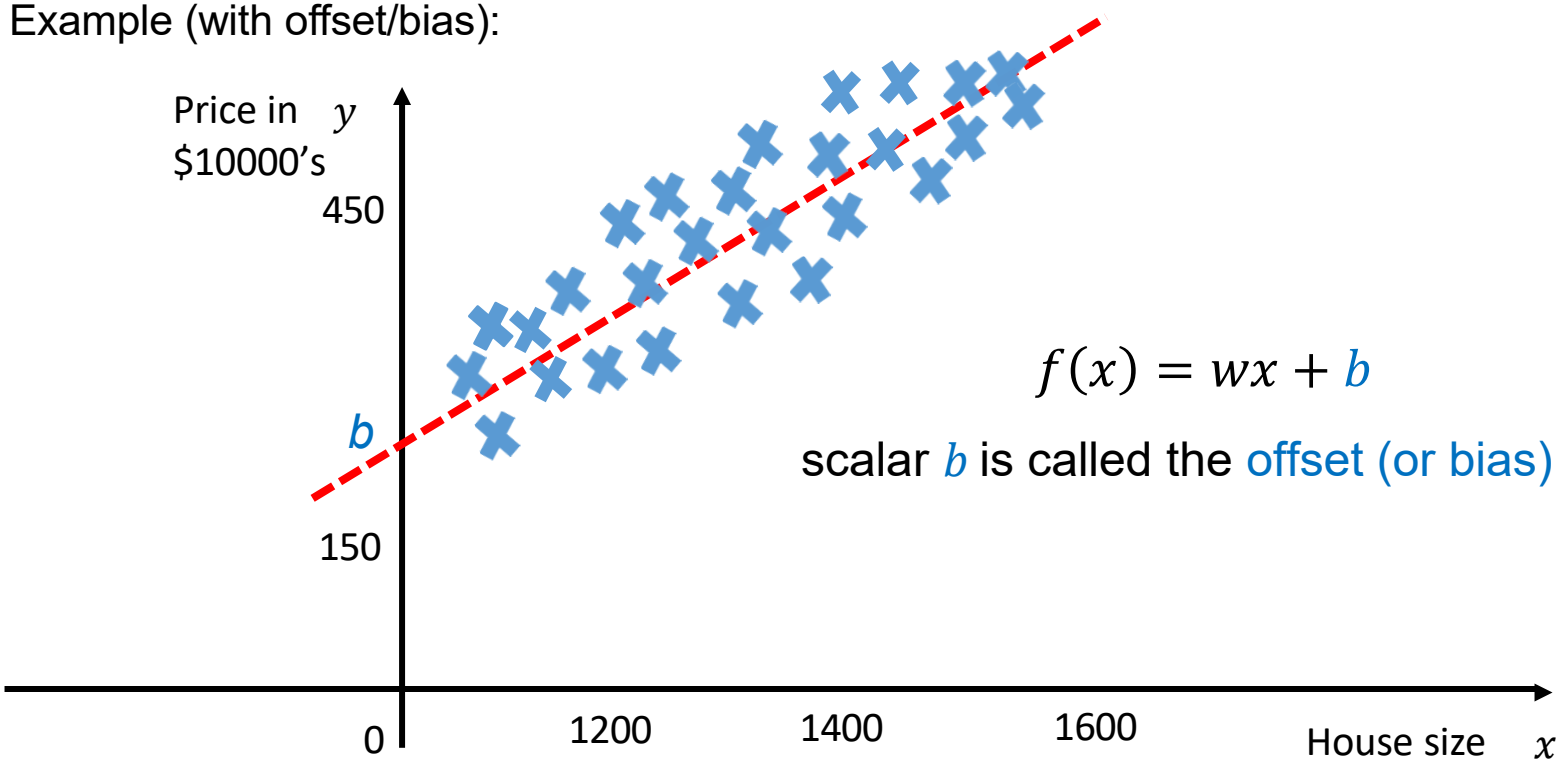
Target: find w

Quick question: Do you see any limitations of the model above?

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example

Example (with offset/bias):



$$f(x) = wx + b$$

scalar b is called the offset (or bias)

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Definition

Problem Statement: To predict the unknown y for a given $\mathbf{x}$ **(testing)**

- We have a collection of labeled examples (used for **training**)
  - m is the size of the collection
  - $\mathbf{x}_i$ is the d-dimensional feature vector of example $i = 1, \ldots, m$ (input)
  - y_i is a real-valued target (1-D)

- We want to build a model $f_{\mathbf{w},b}(\mathbf{x})$ as a linear combination of features of example $\mathbf{x}$:
  $f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{x}^T\mathbf{w} + b = \mathbf{w}^T\mathbf{x} + b$
  where $\mathbf{w}$ is a d-dimensional vector of parameters and b is a real number.
- The notation $f_{\mathbf{w},b}$ means that the model f is parametrized by two values: $\mathbf{w}$ and b

To be learned!

Ref: [Book4] Stephen Boyd and Lieven Vandenberghe, "Introduction to Applied Linear Algebra", Cambridge University Press, 2018 (chp.14)

© Copyright EE, NUS. All Rights Reserved.

Quick Question 3

PollEv.com/shaojing

Consider fitting a linear model $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ to data. Which of the following best explains why we include the bias/offset term b?

A) To allow the model to fit data that does not pass through the origin, enabling it to learn an arbitrary offset in the data.

B) To improve model performance by manually assigning an offset/bias term.

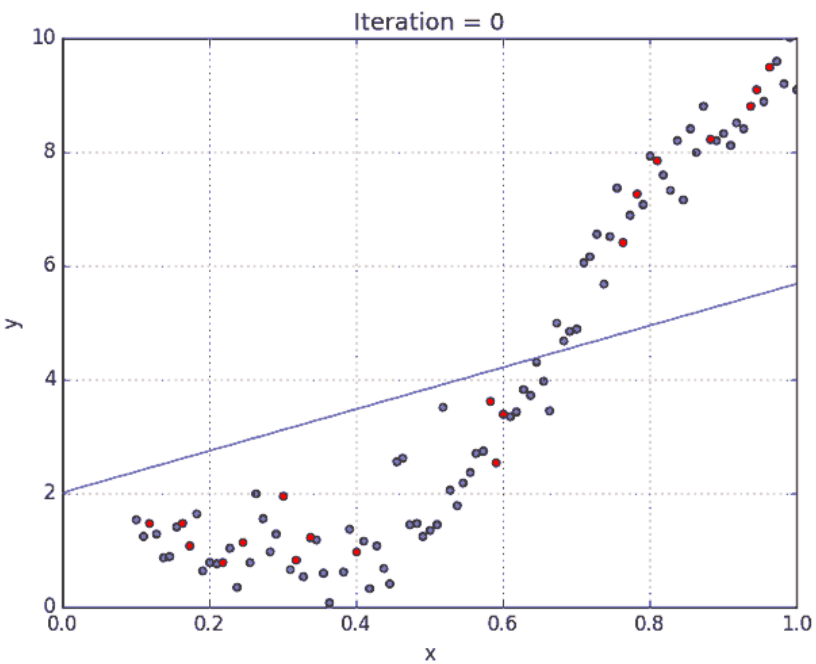
C) To improve model performance by increasing the number of input features.

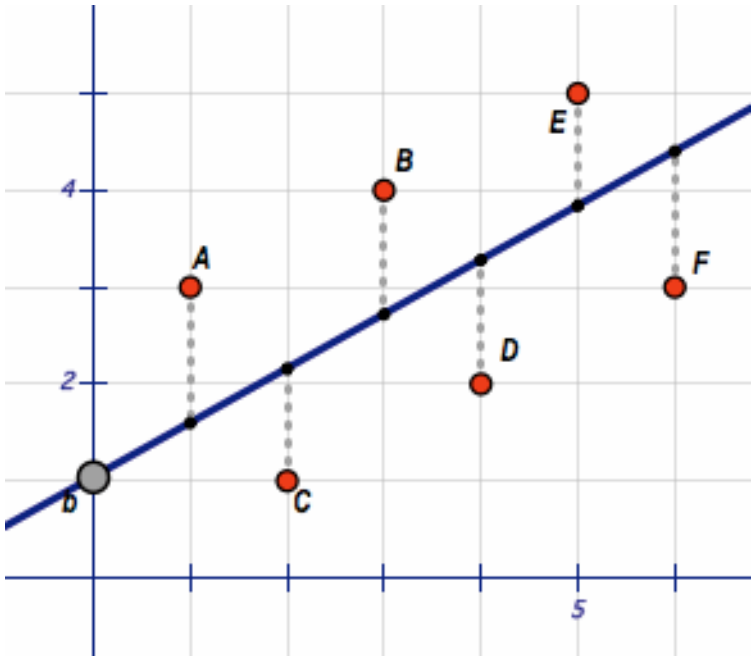
D) To make the model nonlinear.

© Copyright EE, NUS. All Rights Reserved.

Quick Question 3

Consider fitting a linear model $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ to data. Which of the following best explains why we include the bias/offset term b?





Source: https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd
https://commons.wikimedia.org/wiki/File:Least_Squares.gif

© Copyright EE, NUS. All Rights Reserved.

Quick Question 3

Consider fitting a linear model $f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ to data. Which of the following best explains why we include the bias/offset term b?

A) To allow the model to fit data that does not pass through the origin, enabling it to learn an arbitrary offset in the data. ✓ ?

- The bias allows the model to shift the line or plane, so it is not forced to pass through the origin and can better fit the data.

B) To improve model performance by manually assigning an offset/bias term. ✗

- The bias is not manually assigned.
- It is learned automatically from the data during training.

C) To improve model performance by increasing the number of input features. ✗

- Input features come from the data (e.g., age, height, test score).
- The bias is not an input feature, it is constant to be learned

D) To make the model nonlinear. ✗

- Adding a bias does not make the model nonlinear.
- The model remains a linear function of the inputs.

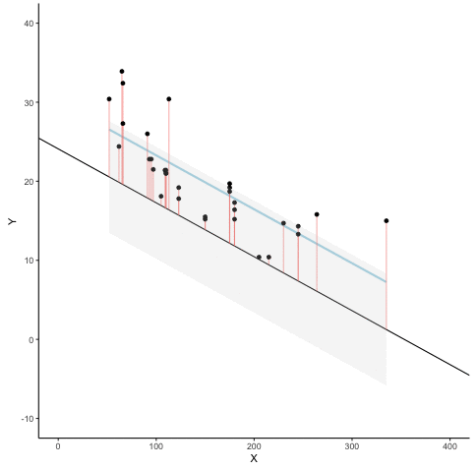
In mathematics, a strict linear function passes through the origin ($y = \mathbf{x}^\top\mathbf{w}$) ,but in machine learning, the linear function concept usually includes the affine form ($y = \mathbf{w}^T\mathbf{x} + b$), where the bias allows the line or plane to shift and better fit the data.

© Copyright EE, NUS. All Rights Reserved.

Quick Question 3: Takeaway

Why always including a bias in linear regression

- The bias allows the model to shift the line or plane so it does not have to pass through the origin.
- It helps the model fit the data more accurately.
- The bias is learned automatically from the data during training.
- It is not an input feature and does not add new information.
- Including a bias does not change the model's linearity.



Always include a bias term!

Source: https://www.crumplab.com/statistics/13-Gifs.html

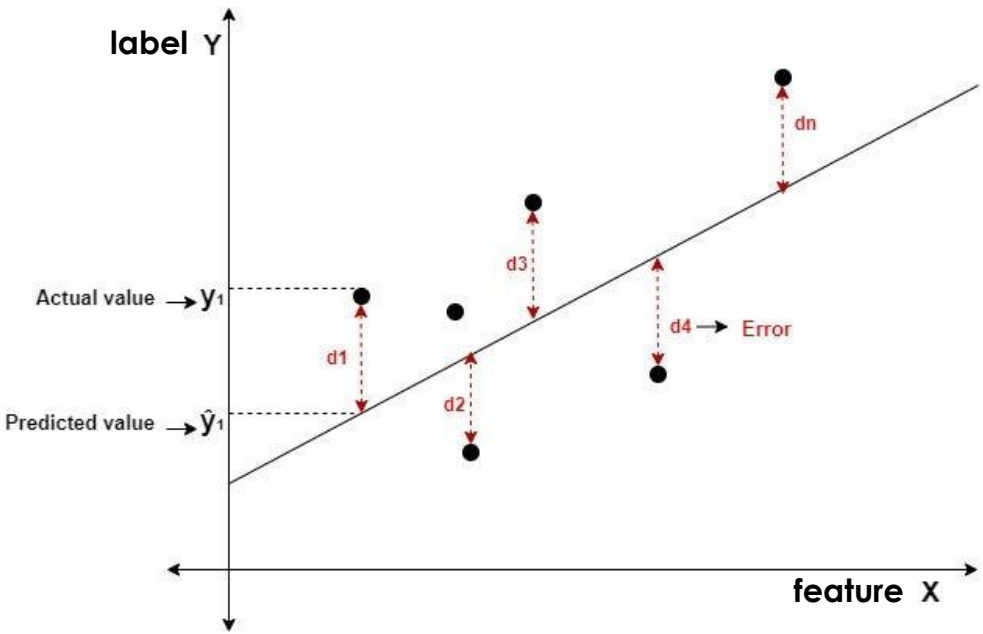
© Copyright EE, NUS. All Rights Reserved.

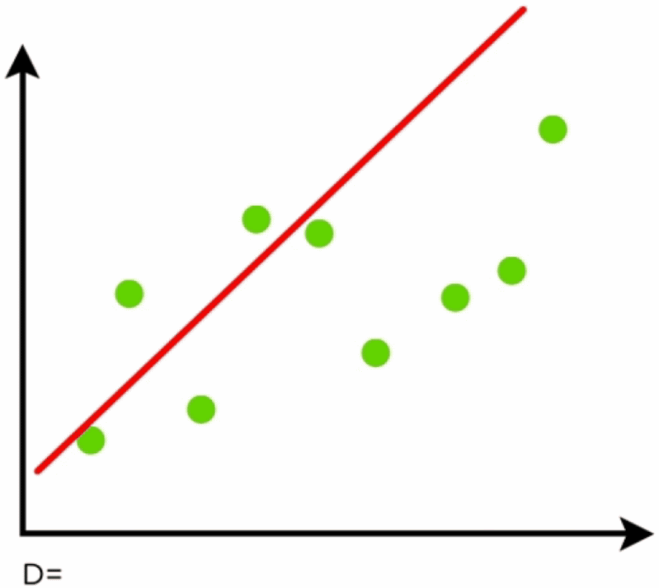
Linear Regression: Learning

Goal: Find weights $\mathbf{w}$ and b in the function $\boldsymbol{f}(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ that best fit the data.

Method: Minimize the prediction error $(\hat{\boldsymbol{y}} - \boldsymbol{y})^2$

The hat in $\hat{y}$ is to indicate that this is an estimated output.





Source: https://blog.gopenai.com/linear-regression-understanding-the-basics-1b1511499911
https://medium.com/@novus_afk/linear-regression-an-overview-13d37a6bc4dd

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Learning

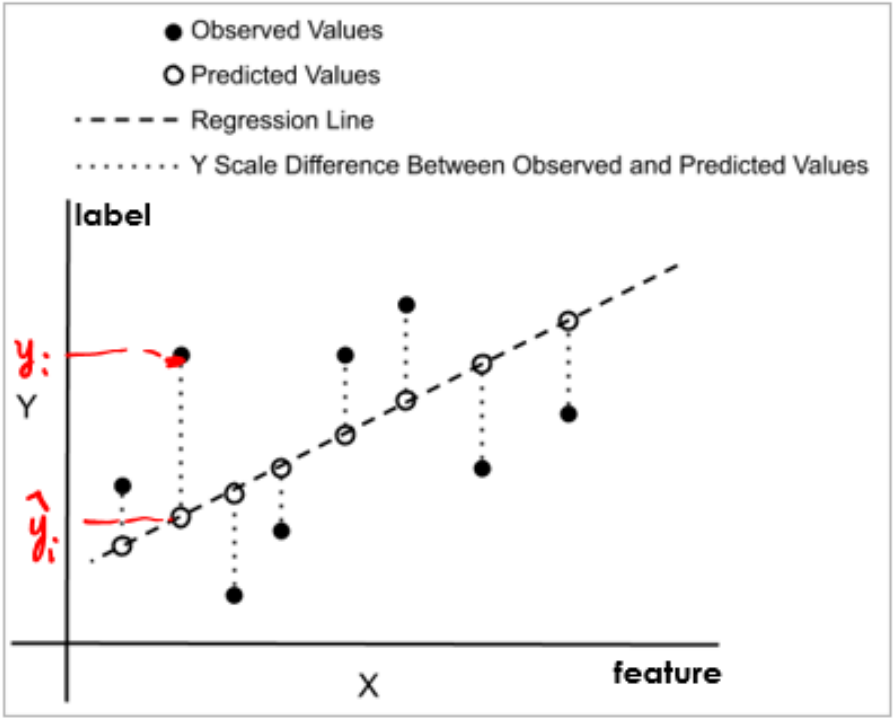
Goal of linear regression: minimize $J(w)$

Learning objective function

- To find the optimal values for $\mathbf{w}^*$ and b^* which **minimizes** the following expression:

$$J(w) = \frac{1}{m}\sum_{i=1}^{m}(\overset{\hat{y}_i}{f_{\mathbf{w},b}(\mathbf{x}_i)} - \mathrm{y}_i)^2$$

m: number of samples



- In mathematics, the expression we minimize or maximize is called an **objective function**, or, simply, an **objective**
- In ML, we use slightly different terms: **loss function** & **cost function** (see next slide)

Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (chp3.1.2)

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Loss Function & Cost Function

Loss Function: Measures error for **a single data point**

$$L_i = (\hat{y}_i - y_i)^2$$

Squared error loss

- $\hat{y}_i = \mathbf{x}_i^\top \mathbf{w} + b$
- Captures how far the prediction is from the true value

Cost Function: Measures **overall error across all data points**

$$J(\mathbf{w}, b) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

Mean Squared Error (MSE)

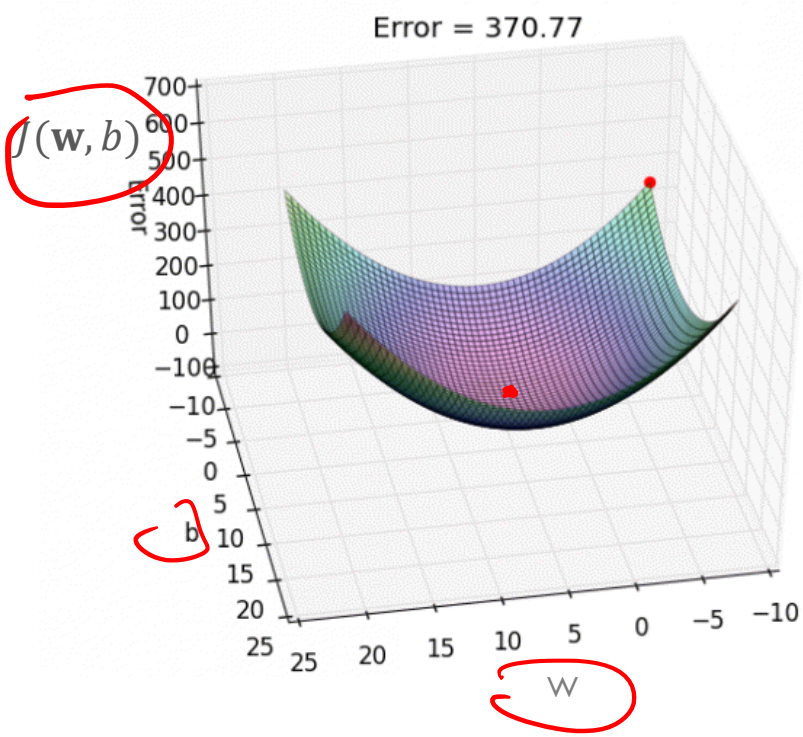
- Average of **all individual** losses
- **Goal of linear regression:** Minimize this cost function to learn the parameters $\mathbf{w}$ and b

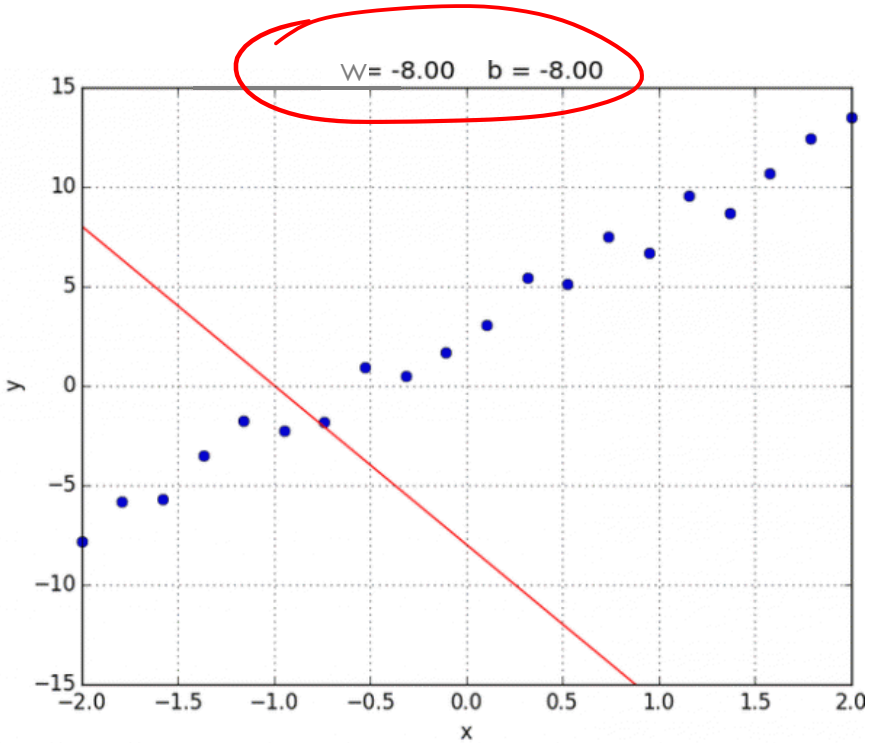
Ref: [Book1] Andriy Burkov, "The Hundred-Page Machine Learning Book", 2019 (chp3.1.2)

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Learning Visualization

Goal of linear regression: minimize $J(w)$





Source: https://medium.com/@yennhi95zz/3-understanding-the-cost-function-in-linear-regression-for-machine-learning-beginners-ec9edeecbdde

© Copyright EE, NUS. All Rights Reserved.

© Copyright EE, NUS. All Rights Reserved.

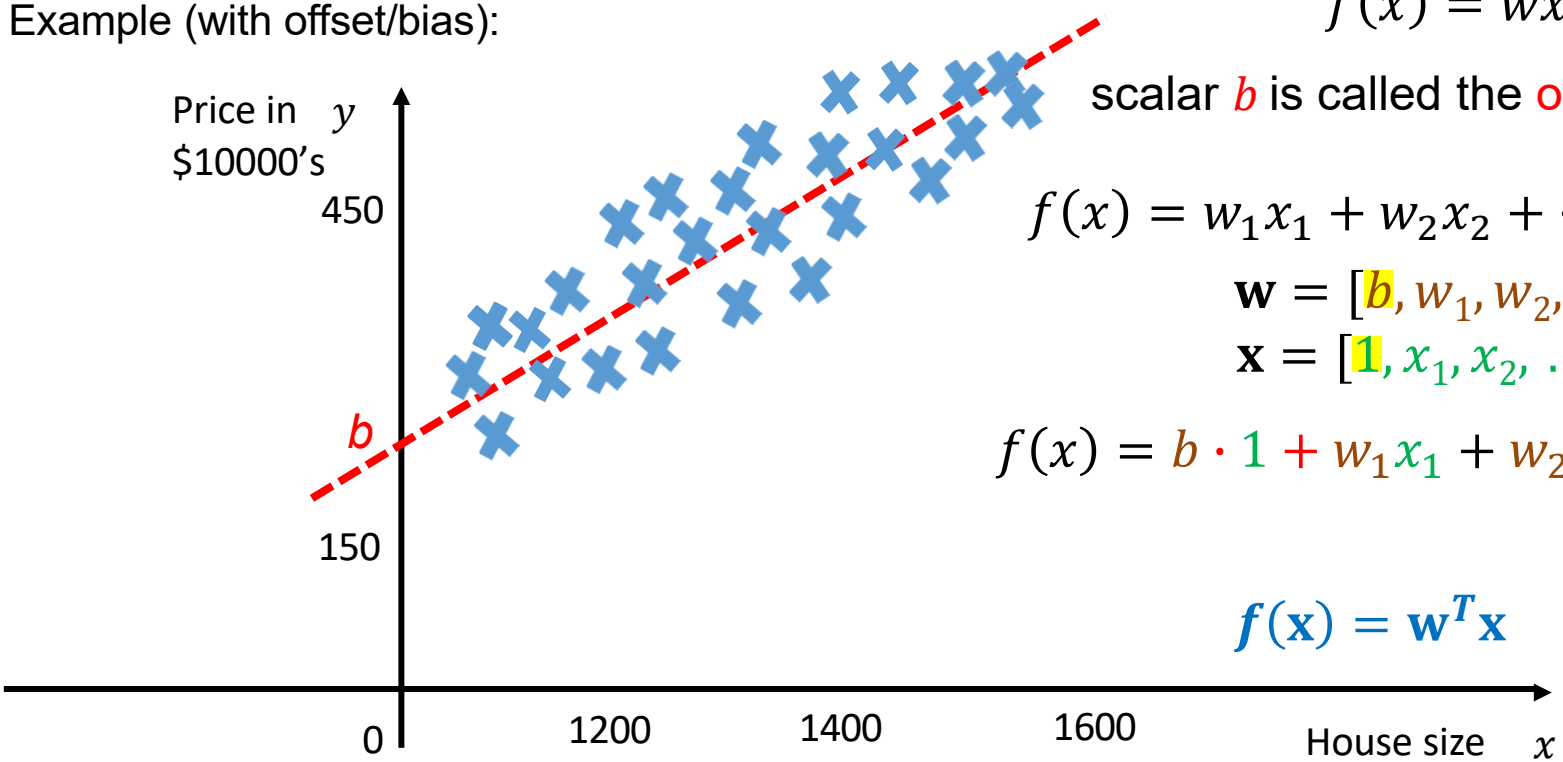
Linear Regression: Simplified Notation

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

Example (with offset/bias):

$$f(x) = wx + b$$

scalar b is called the offset (or bias)



$$f(x) = w_1x_1 + w_2x_2 + \cdots w_dx_d + b$$

$$\mathbf{w} = [b, w_1, w_2, \ldots w_d]$$

$$\mathbf{x} = [1, x_1, x_2, \ldots x_d]$$

$$f(x) = b \cdot 1 + w_1x_1 + w_2x_2 + \cdots w_dx_d$$

$$\boldsymbol{f}(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$$

Simplified notation: absorbing the offset/bias term b into the vector $\mathbf{w}$

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Simplified Notation

$$\mathbf{X}\mathbf{w} = \mathbf{y}$$

← Keep this format while incorporating a bias term.

2

3

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & \dots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \dots & x_{m,d} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix},$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

w_0 → bias

- The data matrix $\mathbf{X} \in \mathcal{R}^{m \times (d+1)}$ and the target vector $\mathbf{y} \in \mathcal{R}^m$ are given
- The unknown vector of parameters $\mathbf{w} \in \mathcal{R}^{(d+1)}$ is to be learnt.

bias

bias

In practice, we typically include a bias term to improve model performance.

Simplified notation: absorbing the offset/bias term b into the vector $\mathbf{w}$

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Learning

$\mathbf{X}\mathbf{w} = \mathbf{y}, \quad \mathbf{X} \in \mathcal{R}^{m\times(d+1)}, \mathbf{w} \in \mathcal{R}^{d+1}, \mathbf{y} \in \mathcal{R}^{m}$, $\mathbf{w}$: to be learned

m: no. of samples, *d*: no. of feature dimensions

Note: when $\boldsymbol{f}_{\mathbf{w}}(\mathbf{X}) = \mathbf{X}\mathbf{w}$, then cost function

$J(\mathbf{w}) = \sum_{i=1}^{m}(f_{\mathbf{w}}(\mathbf{x}_i) - \mathrm{y}_i)^2 = \| \mathbf{X}\mathbf{w} - y \|^2 = (\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y})$.

Objective: To find the best $\hat{\mathbf{w}}$ to minimize the cost function $J(\mathbf{w})$

$$
\begin{aligned}
J(\mathbf{w}) &= (\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y}) \\
&= (\mathbf{w}^T\mathbf{X}^T - \mathbf{y}^T)(\mathbf{X}\mathbf{w} - \mathbf{y}) \\
&= \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{w}^T\mathbf{X}^T\mathbf{y} - \mathbf{y}^T\mathbf{X}\mathbf{w} + \mathbf{y}^T\mathbf{y} \\
&= \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} - 2\mathbf{y}^T\mathbf{X}\mathbf{w} + \mathbf{y}^T\mathbf{y}
\end{aligned}
$$

scalar

Note: In the previous slides, we wrote $J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}(f_{\mathbf{w},b}(\mathbf{x}_i) - \mathrm{y}_i)^2$ *. The normalization factor* $\frac{1}{m}$ *can be omitted here as it does not affect the optimization.*

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Learning & Prediction

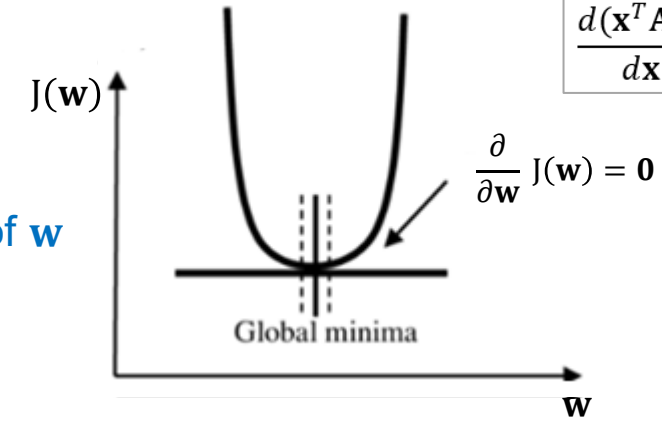
$$\frac{d(\mathbf{a}^T\mathbf{x})}{d\mathbf{x}} = \mathbf{a}$$

$$\frac{d(\mathbf{x}^T\mathbf{A}\mathbf{x})}{d\mathbf{x}} = (\mathbf{A}+\mathbf{A}^T)\mathbf{x}$$

$$\mathrm{J}(\mathbf{w}) = \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} - 2\mathbf{y}^T\mathbf{X}\mathbf{w} + \mathbf{y}^T\mathbf{y}$$

$w \rightarrow \arg\min J(w)$

A convex function of $\mathbf{w}$



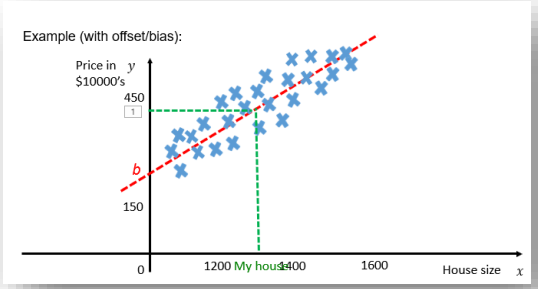
$$\frac{\partial}{\partial \mathbf{w}}\mathrm{J}(\mathbf{w}) = \mathbf{0}$$

To find $\hat{\mathbf{w}}$ that minimizes $\mathrm{J}(\mathbf{w})$, differentiate $\mathrm{J}(\mathbf{w})$ with respect to $\mathbf{w}$ and setting the result to **0**: $\frac{\partial}{\partial \mathbf{w}}\mathrm{J}(\mathbf{w}) = \mathbf{0}$

$$\frac{\partial}{\partial \mathbf{w}}(\mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} - 2\mathbf{y}^T\mathbf{X}\mathbf{w} + \mathbf{y}^T\mathbf{y}) = \mathbf{0}$$

$$\Rightarrow 2\mathbf{X}^T\mathbf{X}\mathbf{w} - 2\mathbf{X}^T\mathbf{y} = \mathbf{0}$$

$$\Rightarrow \mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$$



If $\mathbf{X}^T\mathbf{X}$ is invertible, then

Learning/training: $\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$

Least squares solution ($\hat{\mathbf{w}}$ minimizes the sum of squared errors)

Prediction/testing: $\hat{f}_{\mathbf{w}}(\mathbf{X}_{new}) = \mathbf{X}_{new}\hat{\mathbf{w}}$

Refer to slide #30 for common gradient formulae

© Copyright EE, NUS. All Rights Reserved.

Recap: Solutions of L.E.S

For $\mathbf{X} \in \mathcal{R}^{m\times(d+1)}$ and $\widetilde{\mathbf{X}} = [\mathbf{X}\ \mathbf{y}] \in \mathcal{R}^{m\times(d+2)}$, the system $\mathbf{X}\mathbf{w} = \mathbf{y}$

	Even (m = d)	Over (m > d)	Under (m < d)
i) $\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) = d$	Unique sol: $\hat{\mathbf{w}} = \mathbf{X}^{-1}\mathbf{y}$	Unique sol: $\hat{\mathbf{w}} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}$	--
ii) $\text{rank}(\mathbf{X}) < \text{rank}(\widetilde{\mathbf{X}})$	No sol	No sol in general, but when left inv $\mathbf{X}$ exists, least-squares sol (approx.): $\hat{\mathbf{w}} = (\mathbf{X}^{\mathrm{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathrm{T}}\mathbf{y}$	No sol
iii) $\text{rank}(\mathbf{X}) = \text{rank}(\widetilde{\mathbf{X}}) < d$	Infinitely many sol*	Infinitely many sol*	Infinitely many sol in general, but when right inv $\mathbf{X}$ exists, least-norm sol: $\hat{\mathbf{w}} = \mathbf{X}^{\mathrm{T}}(\mathbf{X}\mathbf{X}^{\mathrm{T}})^{-1}\mathbf{y}$

Learning a linear regression model can be formulated as solving an L.E.S

Least-squares regression learns the model by finding the best approximate solution to $\mathbf{X}\mathbf{w} \approx \mathbf{y}$.

*When the system is under-determined, it selects the least-norm solution among infinitely many solutions.

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example 1 (Training)

Dataset $\{(x_i, y_i)\}_{i=1}^{m}$ includes the samples:

$xw = y$

$$\{x = -9\} \rightarrow \{y = -6\}$$
$$\{x = -7\} \rightarrow \{y = -6\}$$
$$\{x = -5\} \rightarrow \{y = -4\}$$
$$\{x = 1\} \rightarrow \{y = -1\}$$
$$\{x = 5\} \rightarrow \{y = 1\}$$
$$\{x = 9\} \rightarrow \{y = 4\}$$

$$\underbrace{\begin{bmatrix} 1 & -9 \\ 1 & -7 \\ 1 & -5 \\ 1 & 1 \\ 1 & 5 \\ 1 & 9 \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} w_0 \\ w_1 \end{bmatrix}}_{\mathbf{w}} = \underbrace{\begin{bmatrix} -6 \\ -6 \\ -4 \\ -1 \\ 1 \\ 4 \end{bmatrix}}_{\mathbf{y}}$$

bias

Quick question: Why do we include the "1"s here?

This set of linear equations has no exact solution.

However, as $\mathbf{X}^T\mathbf{X}$ is invertible, we have the **least square approximation:**

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

$$= \begin{bmatrix} 6 & -6 \\ -6 & 262 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ -9 & -7 & -5 & 1 & 5 & 9 \end{bmatrix} \begin{bmatrix} -6 \\ -6 \\ -4 \\ -1 \\ 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix}$$

w

w_0 (bias)

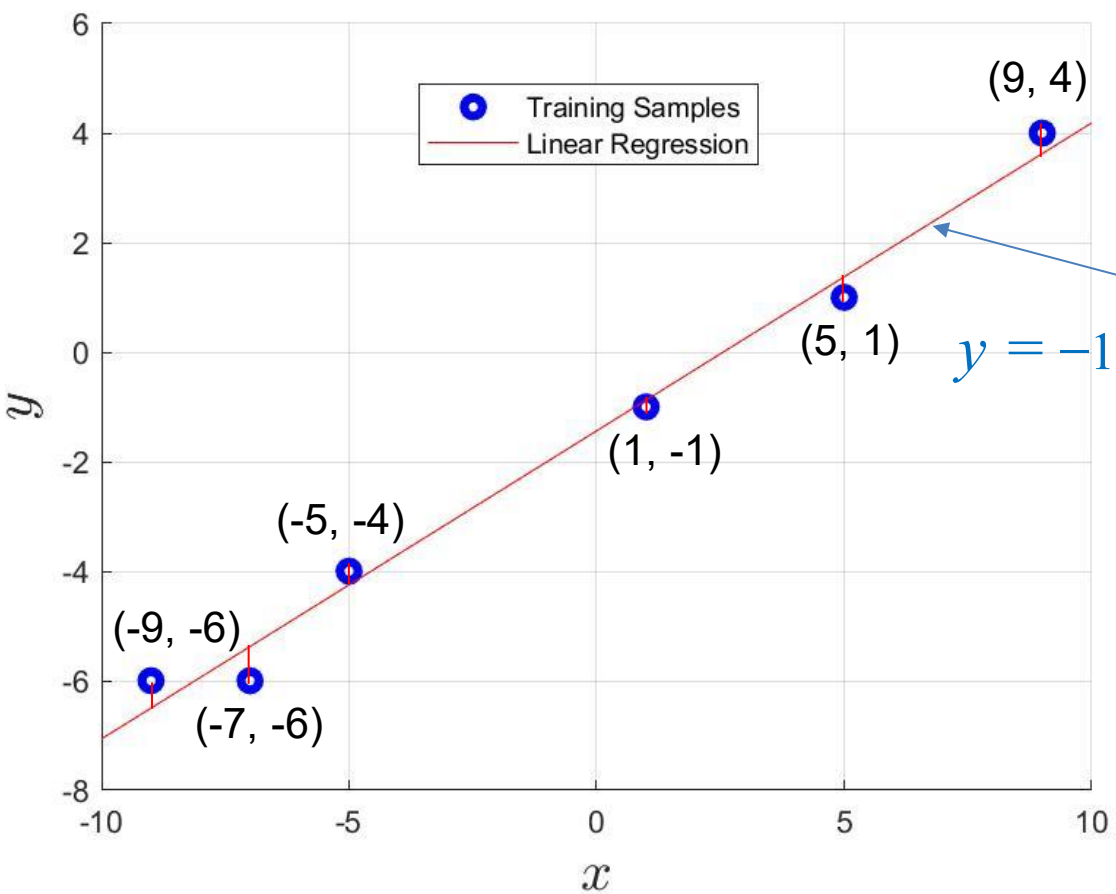
Quick question: What is the value of the bias term?

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example 1 (Training)

We can plot the points and the regression line:

$\{x = -9\} \rightarrow \{y = -6\}$
$\{x = -7\} \rightarrow \{y = -6\}$
$\{x = -5\} \rightarrow \{y = -4\}$
$\{x = 1\} \rightarrow \{y = -1\}$
$\{x = 5\} \rightarrow \{y = 1\}$
$\{x = 9\} \rightarrow \{y = 4\}$



$y = -1.4375 + 0.5625x$

$$\mathbf{Xw} = \mathbf{y}$$

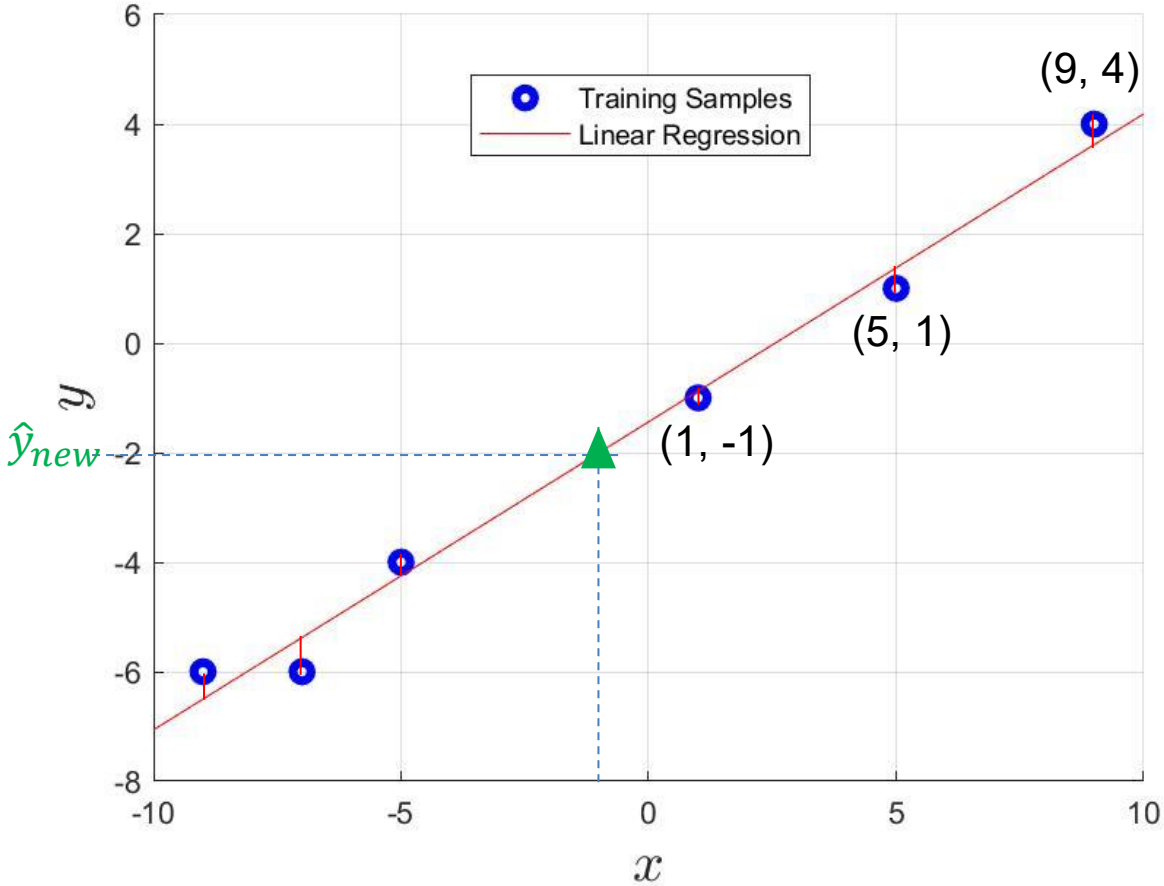
$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} = \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix}$$

Linear Regression on one-dimensional samples

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example 1 (Prediction)

Suppose we want to predict the value of y_{new} when $\mathbf{x}_{new} = -1$.
Then we plug $x_{new} = -1$ into the model to get : $\hat{y}_{new} = \mathbf{x}_{new}\hat{\mathbf{w}}$



Linear Regression on one-dimensional samples

$$\mathbf{Xw} = \mathbf{y}$$

$$\hat{\mathbf{w}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} = \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix}$$

bias

$$\hat{y}_{new} = \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} -1.4375 \\ 0.5625 \end{bmatrix} = -2$$

x_{new} @ $\hat{w}$

Remember to append 1s for both *training* and *testing* data!

Python demo 1

© Copyright EE, NUS. All Rights Reserved.

© Copyright EE, NUS. All Rights Reserved.

Agenda

- Set and functions
- Derivative and gradient
- Linear regression
- Linear regression with multiple outputs

© Copyright EE, NUS. All Rights Reserved.

Linear Regression with Multiple Outputs: Motivation

Suppose I want to predict a student's study habits and attitude based on their attendance and exam scores.

E.g., 3 samples, 2 feature dimensions, 2 prediction targets

Sample	Attendance % (x_1)	Exam score (x_2)	Study habit (y_1)	Attitude (y_2)
Student 1	95	86	1	1
Student 2	50	75	1	0
Student 3	60	52	0	1
Student s	**85**	**93**	**?**	**?**

$$f: \mathcal{R}^d \rightarrow \mathcal{R}^k$$

© Copyright EE, NUS. All Rights Reserved.

Linear Regression with Multiple Outputs

- For one sample: a linear model $\mathbf{f_W}(\mathbf{x}) = \mathbf{x}^T\mathbf{W}$
- For m samples: $\mathbf{F_W}(\mathbf{X}) = \mathbf{XW} = \mathbf{Y}$

Vector function

Sample 1 $\rightarrow$

$\vdots$

Sample m $\rightarrow$

$$= \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_m^T \end{bmatrix} \mathbf{W} = \begin{bmatrix} 1 & x_{1,1} & \cdots & x_{1,d} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m,1} & \cdots & x_{m,d} \end{bmatrix} \underbrace{\begin{bmatrix} w_{0,1} & \cdots & w_{0,h} \\ w_{1,1} & \cdots & w_{1,h} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,h} \end{bmatrix}}_{h}$$

Sample 1's output $\rightarrow$

$\vdots$

Sample m's output $\rightarrow$

$$= \underbrace{\begin{bmatrix} y_{1,1} & \cdots & y_{1,h} \\ & \vdots & \\ y_{m,1} & \cdots & y_{m,h} \end{bmatrix}}_{h} \Big\} m$$

To be learned!

$$\mathbf{X} \in \mathcal{R}^{m\times(d+1)}, \mathbf{W} \in \mathcal{R}^{(d+1)\times h}, \mathbf{Y} \in \mathcal{R}^{m\times h}$$

© Copyright EE, NUS. All Rights Reserved.

Linear Regression with Multiple Outputs

Objective function: $\mathrm{J}(\mathbf{W}) = \sum_{i=1}^{m} \|\mathbf{f}_{\mathbf{W}}(\mathbf{x}_i) - \mathbf{y}_i\|^2$

In matrix notation, this can be written compactly as:

$Y \rightarrow Y$

$$\mathrm{J}(\mathbf{W}) = \| \mathbf{XW} - \mathbf{Y} \|^2$$
$$= \mathrm{trace}[(\mathbf{XW} - \mathbf{Y})^T(\mathbf{XW} - \mathbf{Y})]$$

The trace is a linear algebra operation that sums the diagonal elements of a square matrix. $\mathrm{trace}(\mathbf{A}) = \sum_{k=1}^{n} a_{kk}$, where a_{kk} are the diagonal entries of $\mathbf{A}$.

If $\mathbf{X}^T\mathbf{X}$ is invertible, then

Learning/training: $\widehat{\mathbf{W}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$

Prediction/testing: $\hat{\mathbf{F}}_{\mathbf{W}}(\mathbf{X}_{new}) = \mathbf{X}_{new}\widehat{\mathbf{W}}$

Ref: Hastie, Tibshirani, Friedman, "The Elements of Statistical Learning", (2nd ed., 12th printing) 2017 (chp.3.2.4)

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example 2 (Training)

Training set

$\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^{m}$

$$\{x_1 = 1, x_2 = 1\} \rightarrow \{y_1 = 1, \quad y_2 = 0\}$$
$$\{x_1 = -1, x_2 = 1\} \rightarrow \{y_1 = 0, \quad y_2 = 1\}$$
$$\{x_1 = 1, x_2 = 3\} \rightarrow \{y_1 = 2, y_2 = -1\}$$
$$\{x_1 = 1, x_2 = 0\} \rightarrow \{y_1 = -1, y_2 = 3\}$$

$$\begin{array}{ccc} \mathbf{X} & \mathbf{W} & \mathbf{Y} \end{array}$$

first row = bias weights

Bias

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 3 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ w_{3,1} & w_{3,2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -1 \\ -1 & 3 \end{bmatrix}$$

This set of linear equations has no exact solution.

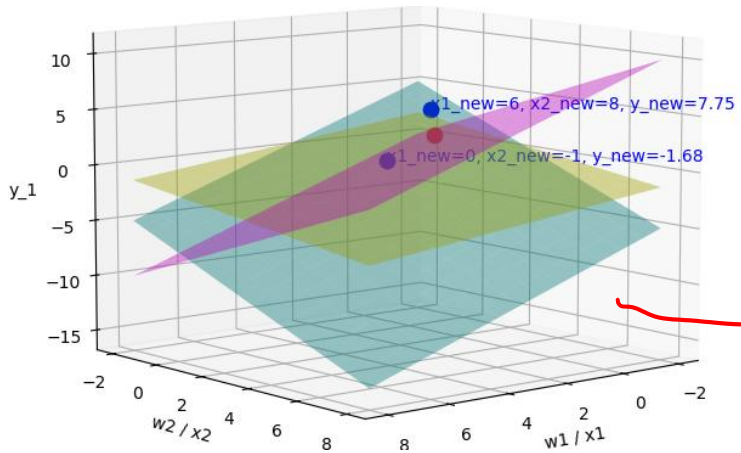
However, as $\mathbf{X}^T\mathbf{X}$ is invertible, we have the **least square approximation:**

$$\hat{\mathbf{W}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

$$= \begin{bmatrix} 4 & 2 & 5 \\ 2 & 4 & 3 \\ 5 & 3 & 11 \end{bmatrix}^{-1} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & 3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & -1 \\ -1 & 3 \end{bmatrix} = \begin{bmatrix} -0.75 & 2.25 \\ 0.1786 & 0.0357 \\ 0.9286 & -1.2143 \end{bmatrix}$$

© Copyright EE, NUS. All Rights Reserved.

Linear Regression: Example 2 (Prediction)



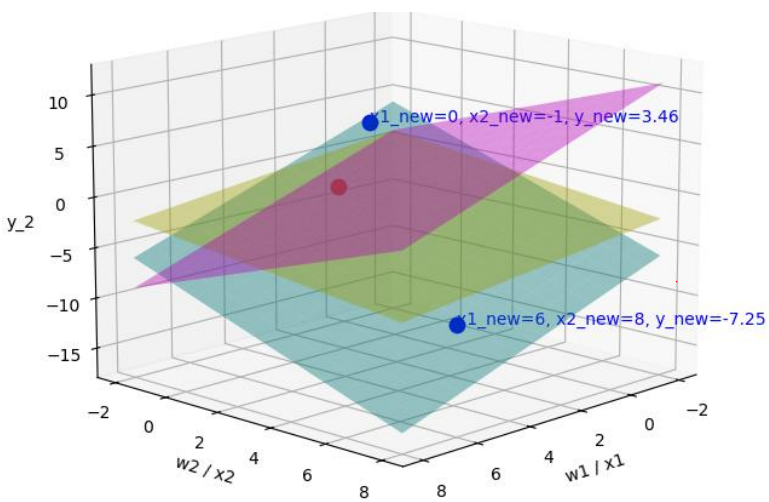
Test set: two new samples

$$\{x_1 = 6, \quad x_3 = 8\} \rightarrow \{y_1 = ?, y_2 = ?\}$$
$$\{x_1 = 0, x_3 = -1\} \rightarrow \{y_1 = ?, y_2 = ?\}$$

y_1 y_2

label label2

$$\hat{\mathbf{Y}}_{new} = \mathbf{X}_{new}\hat{\mathbf{w}}$$



Bias

$$= \begin{bmatrix} 1 & 6 & 8 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} -0.75 & 2.25 \\ 0.1786 & 0.0357 \\ 0.9286 & -1.2143 \end{bmatrix}$$

$$= \begin{bmatrix} 7.75 & -7.25 \\ -1.6786 & 3.4643 \end{bmatrix}$$

Python demo

© Copyright EE, NUS. All Rights Reserved.

Summary

- Linear Regression
  - Objective function, loss function, cost function
  - Least squares solution (LSS), training/learning and testing/prediction
  - Linear regression with multiple outputs

Learning/training (LSS) $\hat{\mathbf{w}} = (\mathbf{X}_{train}^T \mathbf{X}_{train})^{-1} \mathbf{X}_{train}^T \mathbf{y}_{train}$

Prediction/testing $\mathbf{y}_{test} = \mathbf{X}_{test}\, \hat{\mathbf{w}}$

In linear regression, the goal is to find the model parameters that minimize the cost function.

Python packages: numpy, numpy.linalg, and sklearn.metrics (for mean_squared_error)

© Copyright EE, NUS. All Rights Reserved.

References

Linear regression by Dr Andrew Ng:
https://www.youtube.com/watch?v=dLc-lfEEYss&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=9

Cost function by Dr Andrew Ng:
https://www.youtube.com/watch?v=dLc-lfEEYss&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=9

https://www.youtube.com/watch?v=peNRqkfukYY&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=12

https://www.youtube.com/watch?v=bFNz2u0hl9E&list=PLkDaE6sCZn6FNC6YRfRQc_FbeQrF8BwGI&index=13

© Copyright EE, NUS. All Rights Reserved.

Announcement

Assignments

- Q&A link available on Canvas discussion.
- You can also ask anonymously via Canvas discussion.
- Our GAs will respond as soon as possible.

Mid-term Exam

- Covers L1 to L6 (programming related questions only from L4 to L6)
- Trial quiz released before next Tuesday.
- No need to manually compute inverses; software (MATLAB/Python) is allowed.
- Only the final answer needs to be submitted. Method doesn't matter (software or pen & paper).
- Any non-LLM-based approach is acceptable as long as the final answer is correct.
- Any types of calculators (including graphing calculators) are allowed

As requested by the ECE Undergraduate Student Council, we are sharing this invitation: you are welcome to participate in the survey they have organized:

© Copyright EE, NUS. All Rights Reserved.

© Copyright National University of Singapore. All Rights Reserved.